

国产算力平台大模型推理引擎综述

A Survey of Large-Model Inference Engines on Domestic Computing Platforms

张书豪¹, 杜雨枫¹, 刘世峰¹, 马俊豪¹, 邱瑞杰¹, 廖小飞¹, 金海¹
Shuhao Zhang¹, Yufeng Du¹, Shifeng Liu¹, Junhao Ma¹, Ruijie Qiu¹, Xiaofei Liao¹, Hai
Jin¹

¹ 华中科技大学计算机科学与技术学院, 大数据技术与系统国家地方联合工程研究中心,
服务计算技术与系统教育部重点实验室, 集群与网格计算湖北省重点实验室, 武汉
430074

¹School of Computer Science and Technology, Huazhong University of Science and Technology,
National Engineering Research Center for Big Data Technology and System, Services Computing
Technology and System Lab, Cluster and Grid Computing Lab, Wuhan 430074, China

通信作者: 廖小飞, E-mail: xfliao@hust.edu.cn

Corresponding author: Xiaofei Liao, E-mail: xfliao@hust.edu.cn

摘要: 本文面向国产算力平台上的大模型推理系统, 不把综述写成项目名录, 而是围绕“生态格局与路线分化—分析骨架与核心架构—基于骨架的路线比较—由路线比较收束出的关键挑战—未来收敛方向”这一主线, 讨论国产芯片、编译栈、运行时与部署环境如何共同重塑推理引擎的设计边界。本文以近年可交叉核验的公开证据为基础, 综合系统论文与技术报告、官方文档与开源仓库、标准或白皮书, 以及少量仅用于补充真实摩擦点的社区部署记录; 对仅见于单次演示、新闻稿或未公开复现链路的材料, 不直接据此给出性能与成熟度结论。基于这一证据边界, 本文首先梳理当前公开生态为何分化为开源主干复用增强路线、平台一体化方案与国产原生/独立路线三类, 继而以统一抽象、统一指标与可观测证据链为底座, 以执行与通信、状态治理、压缩与成本协同三条主路径为分析骨架, 比较不同路线究竟把复杂度主要压在 backend、platform runtime、service state machine 还是 toolchain/control plane。进一步地, 本文将平台异构、复杂任务负载、压缩收益不稳定、上游快速演进与评测体系失真等问题收敛为若干结构性挑战, 强调国产推理系统的真正竞争点不在局部算子是否足够快, 而在平台能力能否沉淀为契约、运行时状态能否被统一治理、成本与交付能否进入同一闭环。最后, 本文结合 vLLM-HUST 这类基于 vLLM 的本土增强实践以及 vLLM-Ascend、MindIE、LMDeploy、Chitu、xLLM 等对象, 讨论平台抽象、状态中心化运行时、控制面治理与模型结构反向塑造系统边界的未来方向, 以期国产推理系统的比较分析与架构演进提供更可复核的讨论框架。

关键词: 国产算力平台; 大模型推理; 推理引擎; 运行时; 软硬协同

Abstract: This survey examines large-model inference systems on domestic computing platforms through a writing spine that moves from ecosystem divergence to analytical framing, route comparison, structural challenges, and future convergence. Rather than enumerating projects, it builds its discussion on cross-checkable public evidence from system

papers and technical reports, official documentation and open-source repositories, standards or white papers, and a limited set of community deployment records used only to expose recurring engineering frictions. Materials that appear only in one-off demos, news releases, or non-reproducible validation reports are not used as standalone evidence for performance or maturity claims. On top of this evidence boundary, the survey explains why the public ecosystem has split into three broad routes: upstream-core reuse and enhancement, vertically integrated platform stacks, and domestic-native or standalone systems. It then adopts an analytical scaffold built on a unified abstraction, metric, and observability base together with three main paths for execution and communication, state governance, and compression-cost co-optimization, and uses this scaffold to compare where different routes place their primary complexity: backend adaptation, platform runtime, service state machine, or toolchain and control plane. Based on this comparison, the survey further distills structural challenges involving hardware heterogeneity, complex workloads, unstable compression gains, rapid upstream evolution, and evidence gaps between benchmarks and production. The core argument is that the decisive issue is not isolated kernel speed, but whether platform capabilities can be turned into stable contracts, runtime states can be governed coherently, and cost, evaluation, and delivery can be brought into one engineering loop. Drawing on localized enhancement practices such as vLLM-HUST together with systems including vLLM-Ascend, MindIE, LMDeploy, Chitu, and xLLM, it finally outlines future directions around platform abstraction, state-centric runtimes, control-plane governance, and the continuing feedback from model evolution into system boundaries.

Keywords: domestic computing platforms; large-model inference; inference engine; runtime systems; hardware-software co-design

1 引言

大模型应用的快速扩张，正把推理系统从单机实验环境推向面向生产的高并发服务平台。随着国产 AI 加速芯片、服务器整机与集群软件生态逐步成熟，围绕国产算力构建高性能、可维护、可扩展的推理引擎，已成为学界和产业界共同关注的方向。但“国产算力推理引擎”并不是把既有 serving 框架迁移到另一类设备上的简单重复。相较通用硬件环境，国产平台往往同时叠加设备能力离散、编译链约束差异、运行时治理复杂和交付路径多样等因素，使许多原本可以局部吸收的问题迅速上升为系统级问题。平台边界、运行时状态与工程闭环一旦同时变化，推理系统的设计重心也会随之迁移。与训练系统相比，推理引擎更关心动态批处理、显存复用、流式时延、长上下文，以及工具调用、多模态和结构化输出下的稳定性；对国产平台而言，还要额外处理硬件后端差异、算子完备性、编译运行时兼容性，以及上游框架快速演进带来的维护压力。

近年来，以 vLLM 为代表的推理框架通过 PagedAttention、连续批处理与高效内存管理显著改写了大模型服务系统的设计范式，SGLang 等系统则进一步强调请求编排、结构化控制流和复杂工作负载下的执行效率 [1–3]。通用 serving 文献也表明，推理系统的研

究重心正在持续外移：Orca 与 Sarathi-Serve 说明，迭代级调度、连续批处理和 chunked prefill 已成为高吞吐服务的基础组织方式 [4, 5]；DistServe 与 Splitwise 表明，prefill 和 decode 的阶段解耦正在从局部优化上升为整体 goodput 设计问题 [6, 7]；XGrammar 与 ElasticMM 则进一步把演进推向结构化执行与多模态并行，说明推理引擎正在从 token 生成器演变为状态密集型运行时 [8, 9]。这意味着，当推理系统进入国产芯片、国产编译运行时和本土部署环境后，关键问题会迅速从单一硬件后端优化转向统一框架抽象与本土设备能力的持续对齐。此时，调度、缓存、多模态和结构化执行的可迁移性、可维护性与稳定性，往往比单点优化更重要。从产业侧看，国产算力推理引擎面向的也早已不只是学术基准测试，而是在线问答、代码助手、政企知识服务、具身智能中间层和 Agent 工作流等多种 AGI4S 场景；这些场景既要求较高的吞吐密度和资源利用率，也要求流式首 token 时延、上下文切换开销、错误恢复能力和服务可观测性达到生产标准。因此，国产推理引擎的竞争焦点正从“能否完成基础部署”转向“能否在真实业务下稳定、高效且可持续演进地运行”。

在这一背景下，开源主干复用增强路线、平台一体化方案和国产原生/独立路线虽然都在解决国产部署问题，但三者分化的根源并不只是优化手段不同，更在于对系统边界和演进节奏的假设不同：前者强调上游复用，并允许在共享主干外侧持续注入平台与场景增强，其中既包括直接围绕主线后端边界展开的适配，也包括以 vLLM-HUST 为代表的本土增强实践；中者强调平台统一性交付；后者则更侧重围绕自研状态机、缓存治理和部署闭环组织系统能力。相较训练侧更容易用 FLOPS、集群规模等指标描述阶段进展，推理侧的核心矛盾更多分布在请求形态、缓存复用、阶段干扰、故障恢复与升级节奏等难以被单一数字概括的维度上。尤其当模型族从纯文本扩展到多模态、语音和 Agent 工作流后，系统能否在较长时间尺度内维持接口兼容、性能稳定和运维可控，往往比一次局部 benchmark 的领先更能决定其工程价值。基于此，全文采用“一个底座、三条主路径”的分析骨架：底座是统一抽象、统一指标和可观测证据链，用来支撑多平台、多版本和多阶段优化条件下的可组合、可比较与可复现；三条主路径分别是执行与通信、状态治理、压缩与成本协同，对应执行骨架收敛、长上下文与高并发下的状态组织，以及单位 token 成本进入系统闭环。在此基础上，后文依次讨论生态格局与路线分化、分析骨架与核心架构、基于骨架的路线比较、由路线比较收束出的关键挑战，以及未来可能的收敛方向。全文重点不在于为某一路线背书，而在于识别哪些能力应沉淀为能力契约，哪些状态必须纳入运行时统一管理，哪些优化适合以插件或外围工具链独立演进，以及哪些 benchmark 足以支撑路线选择。

为使讨论建立在可交叉核验的材料之上，本文还对讨论范围和证据边界作如下限定。首先，本文的“国产算力推理引擎”既包括直接面向国产芯片的原生推理系统，也包括基于主流 serving 主干的国产后端适配、面向多类国产硬件的部署工具链，以及围绕服务状态机、缓存治理和交付闭环组织的独立运行时；但不把单纯的硬件整机、一体机交付页面或应用层封装直接等同于推理引擎本体。其次，本文优先使用四类证据：系统论文与技术报告用于界定机制与设计目标，官方文档与开源仓库用于确认能力边界和工程入口，标准或白皮书用于辅助描述生态收敛趋势，社区部署记录仅作为观察真实摩擦点的补充材料，而不单独支撑性能结论。再次，文中对路线比较主要围绕可演进性、状态治理能力、复杂负载承载能力和交付确定性展开，而不试图在缺少统一复现实验条件的前提下给出跨平台绝对性能排序。本文更关注复杂度被压在哪里、证据是否足以支撑相关判断，

以及系统是否具备可持续演进边界，而不是把不同公开口径简单折算为单一优劣结论。

1.1 研究对象与证据方法

本文的综述对象，限定为近年在公开论文、技术报告、官方文档或开源仓库中能够形成交叉验证的国产算力推理相关系统与工程实践。就时间范围而言，文章重点覆盖大模型 serving 主干与国产后端适配快速演进的近年公开材料，并在必要处回溯少量更早的基础工作，用于交代连续批处理、PagedAttention、阶段解耦和结构化执行等机制来源。就对象纳入而言，只有当某一对象至少能够在“机制说明”“工程入口”“公开实现”三者中满足其中两项时，本文才将其纳入路线比较或章节主线；若某材料仅以新闻稿、发布会口径、单次演示或未公开复现链路的技术验证出现，则只作为生态线索或趋势性观察引用，不直接据此给出成熟度与性能判断。

基于这一原则，本文将证据分为四层使用。第一层是系统论文与技术报告，用于界定执行、状态、压缩和评测等机制问题的研究脉络；第二层是官方文档、源码仓库与安装部署入口，用于确认对象的实际能力边界、组件关系与可进入性；第三层是标准、白皮书与生态报告，用于辅助刻画国产生态的分工变化与标准化趋势；第四层才是社区部署记录和兼容性经验，它们仅用于补充真实工程摩擦点，而不单独支撑横向性能结论。按这一分层，正文中的比较表与路线判断优先回答三类问题：一是复杂度主要被压在何处，二是哪些系统边界已经具有较稳定的公开证据，三是哪些判断仍应保留为条件性的工程观察。这样做的目的，不是把本文伪装为严格元分析，而是把综述中的描述性事实、比较性判断与趋势性推断尽量分层组织，使不同性质的材料不被直接并列。

2 生态格局与路线分化

本章聚焦当前国产推理生态的参与者构成、组织方式及其分化动力，暂不直接展开路线优劣比较。相比“出现了哪些项目”，更关键的是辨认复杂度首先显化的位置、主要承担主体及其交付形态 [10–13]。从这一层观察入手，国产推理引擎的后续比较才能建立在相对稳定的生态事实上，而不是停留在项目罗列上。

据此，当前公开生态大致可归纳为**平台一体化方案**、**开源主干复用增强路线**与**国产原生/独立路线**三类。这里的三分法首先是一种生态归纳，而不是结论先行的价值排序：平台一体化路线围绕厂商基础软件栈组织原生推理与部署体系，强调编译、运行时和服务模板的全栈闭环；开源主干复用增强路线围绕 vLLM、SGLang、LMDeploy 等主流主干展开后端适配、插件扩展与本土能力增强，强调生态兼容与特性跟进；国产原生/独立路线则试图在不依附单一上游主干的前提下重组引擎边界、服务状态机和部署闭环，强调对系统边界和长期演进节奏的自主控制 [14–20]。

这三类路线之所以并存，是因为国产推理系统同时受到模型侧演进、平台侧约束和工程交付三股力量牵引。DeepSeek-V4 所代表的模型-系统协同演进，更多从复杂负载、长上下文和任务形态变化出发，倒逼推理引擎重写执行路径与状态治理；Ascend 一类平台则代表硬件、编译器与基础软件栈对引擎形态的强约束，推动平台一体化或深度适配路线形成 [21]。因此，路线分化不是简单的厂商偏好差异，而是不同参与者围绕生态兼容、平台特化和系统边界控制权作出的工程取舍。

在现实工程中，当前最普遍的形态仍是“主流框架 + 国产硬件插件化适配”。vLLM-

Ascend、vLLM-MLU、vLLM-Kunlun、SGLang Ascend 等项目说明，大量国产适配工作并不是从零重写推理引擎，而是优先复用上游框架已有的连续批处理、KV Cache 管理、OpenAI 兼容接口以及量化、投机解码等能力，再通过后端适配、算子移植和外围治理把这些能力迁移到国产硬件。vLLM-HUST 这类本土增强实践则进一步表明，在复用 serving 主体的同时，平台护栏、场景能力和交付控制面也会成为本地工程的重要投入点 [15, 22–25]。这也说明当前生态首先呈现的是“对象先出现、路线再逐步分化”的现实图景，参与者构成与组织方式本身就构成了后续路线比较的前提。

也正因如此，vLLM 插件化架构成为理解当前国产生态的关键切口。其演进重点是从早期 **侵入式修改 (In-Tree)** 转向 **非侵入式插件 (Out-of-Tree, OOT)**，通过在调度、KV Cache 与算子层抽象统一接口，把大量平台差异压回 plugin backend、attention backend 与 distributed backend 等扩展边界，从而缓解“一版本一修改、上游更新即失效”的维护问题。当前，vLLM OOT 插件架构已成为国产算力适配的事实标准，中国信通院和上海人工智能实验室 DeepLink 生态也在沿这一方向推进统一接口 [26–28]。但插件化并未消除矛盾，而是把问题集中到跨硬件复用、强硬件特性释放和长期维护成本上，主要体现在以下三方面：

- **标准化程度不足**：不同厂商的插件实现仍存在碎片化，难以实现跨硬件的插件复用。
- **硬件特性释放不充分**：MoE 分布式通信、片上内存管理等强硬件相关特性，仍需侵入式修改主干代码才能实现最优性能。
- **维护成本依然较高**：国产硬件适配代码大多维护在厂商独立仓库中，难以进入上游主干，面临长期同步成本。

华为 Ascend 生态是目前公开资料较为完整的代表。一方面，MindIE 作为面向 Ascend 的 AI 推理加速套件，覆盖大模型推理、服务化和 PyTorch 生态适配等能力；另一方面，vLLM-Ascend、SGLang-Kernel-NPU 和 Triton-Ascend 分别从推理框架插件、SGLang 算子库和 Triton 编译后端三个层面补充了开源生态。这说明国产算力推理引擎的竞争点已经从单一模型执行扩展到框架调度、KV Cache、MoE 通信、算子融合、编译优化和服务化接口等完整技术栈。除单一芯片生态外，跨硬件适配层也在快速发展。上海人工智能实验室 DeepLink 生态中的 DLinfer 试图在上层 LLM 推理框架与下层厂商融合算子或图引擎之间定义统一接口，LMDeploy 则通过 PyTorch Engine 和 DLinfer 等路径支持多类国产硬件；百度飞桨 FastDeploy 以 PaddlePaddle 为基础，面向文心等大模型提供推理部署能力，并在官方文档中列出对昆仑芯、Ascend、海光、天数智芯、沐曦、燧原等硬件的适配说明；Xinference、GPUStack 一类服务平台则更侧重统一服务入口、模型编排和多硬件资源管理。它们的共同作用，是减少每个推理框架分别对接每个硬件后端的重复工程成本，并把部署与运维复杂度进一步从引擎本体外移。

表 1: 面向国产算力的大模型推理引擎与适配生态

组织/单位	面向算力	代表项目/引擎	公开形态与技术关注点
华为 Ascend 生态	Ascend NPU, Atlas 推理/训练服务器	MindIE, vLLM-Ascend, SGLang-Kernel-NPU, Triton-Ascend	覆盖厂商推理套件、vLLM 硬件插件、SGLang NPU kernel 与 Triton 后端。技术关注点包括 CANN/torch-npu 适配、服务化推理、MoE kernel、注意力算子、编译后端和硬件插件化接口 [29–32]。

组织/单位	面向算力	代表项目/引擎	公开形态与技术关注点
SGLang 社区与 Ascend 适配	Ascend NPU	SGLang-Kernel-NPU, DeepEP-Ascend	面向 SGLang 的 Ascend NPU kernel 库, 包含专家并行通信、attention、normalization、activation、LoRA 等推理关键算子, 体现了高性能 serving 框架在国产硬件上的 kernel 适配方向 [31]。
上海人工智能实验室 DeepLink / OpenMMLab / InternLM 生态	多类国产加速器, 包括 Ascend、Cambricon、MetaX 等	DLInfer, LMDeploy	DLInfer 面向 LLM 推理框架与厂商算子/图引擎之间的解耦, LMDeploy 则提供 TurboMind 与 PyTorch Engine 两类推理引擎, 并通过构建目标或适配层支持多种国产硬件 [33-35]。
百度飞桨 / 昆仑芯生态	Kunlun XPU, 同时覆盖多种国产硬件	FastDeploy, vLLM-Kunlun	同一生态内部横跨两类路线: 基于 PaddlePaddle/FastDeploy 的原生部署栈更接近平台一体化路线, 强调生产级部署、PD 分离、统一 KV Cache 传输与服务接口; vLLM-Kunlun 则以 vLLM 硬件插件形式支持昆仑 XPU, 属于开源主干适配路线 [14, 15]。
寒武纪	Cambricon MLU	vLLM-MLU, MagicMind, CNServing	vLLM-MLU 基于 vLLM 插件系统在 MLU 硬件上提供大模型推理服务能力; MagicMind 是寒武纪面向 MLU 的推理加速引擎, 负责模型转换、图优化、代码生成和部署 [36, 37]。
摩尔线程	MUSA GPU	vLLM-MUSA, vLLM-MTT / MT-Transformer	vLLM-MUSA 强调与 vLLM 插件体系和 MUSA 软件栈集成; vLLM-MTT 则基于摩尔线程自研 MT-Transformer 后端, 强调面向 Transformer 推理的底层算子融合和定制优化。二者体现了“通用兼容插件”和“高性能原生后端”两条路线 [38, 39]。
沐曦集成电路	MetaX GPU, MACA 软件栈	vLLM-MetaX, LMDeploy/DLInfer 适配	vLLM-MetaX 通过 MACA 类 CUDA 后端接入 vLLM, 目标是在沐曦 GPU 上获得接近 CUDA 生态的开发体验; DLInfer 也将 MetaX 作为其支持的硬件后端之一 [34, 40]。
燧原科技	Enflame GCU, S60/L600 等	vLLM-GCU, FastDeploy-Enflame	vLLM-GCU 面向燧原 GCU 适配 vLLM, 支持量化、Fused MoE、Multi-LoRA、自动 Prefix Cache、Chunked Prefill、Speculative Decoding 等推理特性; FastDeploy 也提供燧原硬件部署说明 [14, 41, 42]。
天数智芯 / DeepSpark 生态	Iluvatar CoreX、智铠/天玑系列	IxRT, IGIE, DeepSparkInference, FastDeploy-Iluvatar	IxRT 和 IGIE 分别面向高性能推理运行时和基于 TVM 的通用推理引擎, DeepSparkInference 提供包括 ChatGLM、Baichuan、Qwen 等模型在内的推理示例; FastDeploy 也提供 Iluvatar CoreX 安装与部署路径 [43, 44]。
海光信息	Hygon DCU, K100-AI 等	FastDeploy-Hygon DCU	官方 FastDeploy 文档给出了在海光 DCU 上部署 ERNIE 系列模型的示例, 说明该方向已有面向大模型推理的公开部署路径。但公开资料更多体现为部署适配和示例, 成熟度仍需通过复现实验进一步判断 [14, 45]。
壁仞科技	Biren SUPA / BR 系列加速器	BIRENSUPA, EngineX-Biren vLLM 适配	壁仞公开开发者生态展示了 BIRENSUPA 软件平台; 同时公开模型社区中存在基于 vLLM OOT 插件的壁仞适配项目。现阶段公开资料相对有限, 当前更适合将其视为待持续跟踪的适配生态, 尚不足以支撑明确性能结论 [46, 47]。
瀚博半导体 / Vastai 生态	Vastai GPU, VACC/VUCA 软件栈	Xinference VACC Vastai 开发者平台	瀚博侧公开资料更多体现为开发者平台和模型生态; Xinference 等服务平台已出现对 Vastai GPU/VACC 的适配支持。该方向可归入国产硬件服务化部署平台生态, 而不是单独的核心推理引擎 [48, 49]。
Xinference / GPUStack 等平台	多类异构 GPU/NPU	Xinference, GPUStack	这类项目位于模型服务和集群管理层, 通常不直接替代 vLLM、SGLang、LMDeploy 等推理引擎, 而是对其进行编排、部署和多硬件管理 [49, 50]。

上述生态实践与技术特征已经勾勒出三类路线在参与主体、接口关系与复杂度承接

方式上的差异轮廓。此处的比较仍停留在生态归位层面，用以说明不同参与者为何会自然汇聚成不同组织方式，而不提前替代后文的技术比较。

表 2：国产推理引擎三类路线的生态特征与复杂度落点概览

对比维度	开源主干复用增强路线	平台一体化路线	国产原生/独立路线
生态接口关系	通常复用主流 serving 接口、模型格式与社区能力，并将国产差异压回后端插件边界	更多围绕单厂商软硬件栈组织编译、运行时与服务接口	接口形态分化较大，是否兼容 OpenAI API、HuggingFace 格式或既有部署接口取决于各自实现
复杂度首先显化的位置	主要落在 plugin backend、版本矩阵和共享主干协同	主要落在 compiler/runtime、服务模板与交付组件	主要落在 service state machine、缓存治理、模型矩阵与部署闭环
复杂任务扩展方式	更多沿主流社区能力演进，并通过插件、护栏和本地治理吸收平台差异	更多依赖平台产品节奏、模型支持矩阵和预置服务组件释放能力	更多依赖自研状态机、调度边界与服务层组织方式显式承接多阶段请求
运行时治理重心	上游框架原生能力与本地插件/护栏的协同边界	版本矩阵、部署模板、监控与运维入口的统一性	状态、缓存、故障恢复和跨阶段迁移边界的自治性
长期维护负担来源	上游接口变化、插件规范演进和后端依赖同步	全栈软件、模型适配与交付模板的持续维护	模型支持、服务接口、后端适配与部署闭环的同时经营
公开材料中更常见的形态	硬件插件仓库、适配分支、本土增强实践与外围治理工具	平台推理套件、服务化栈、镜像与模板化交付系统	独立引擎、部署入口或围绕状态治理组织的运行时工程

该表把三类路线在公开材料中最常显现的接口关系、复杂度落点和维护负担放到同一层观察。第三类也不是内部同质的成熟模板，而是对一组不依附单一上游主干、试图重组系统边界对象的谨慎归纳，其中独立引擎与支撑平台并不简单同构。进一步看，上述生态已呈现出几个较稳定的演进趋势：国产算力适配正从直接修改上游推理框架源码转向硬件插件化和后端解耦，这种方式既降低维护成本，也更便于跟随 vLLM、SGLang 等主流框架的快速演进；厂商原生推理引擎在算子融合、图优化、MoE 通信、低精度量化和特定硬件内存层次优化方面仍保持重要性，百度飞桨/昆仑芯围绕 PaddlePaddle 与 FastDeploy 的部署体系也更接近这一方向；DLInfer、Triton-Ascend 等统一适配抽象层，以及 FastDeploy 这类部署工具链，则说明开源主干复用增强路线也在从单点插件开发走向“统一接口 + 厂商实现”的模式 [10, 11]。

在生态快速演进的同时，几类更稳定的分化压力也已逐渐显现。它们解释了为何单纯沿项目名展开，难以准确描述国产推理生态的真实受力点；相应的系统挑战将在后文结合统一分析骨架进一步展开：

- (1) **生态碎片化与标准化压力：**国产算力生态目前仍存在“一硬件一栈、一框架一适配”的碎片化现象。不同硬件平台的基础软件栈、算子接口与通信语义差异巨大，导致行业重复适配成本极高。这说明后文必须引入统一抽象与统一指标，才能讨论平台差异究竟被吸收到哪一层 [51]。

- (2) **开源复用与边界控制压力**：主流推理框架体系仍由海外社区主导，国产适配多处于“跟随式”状态。如何在深度兼容开源生态的同时维持本地执行架构、状态治理和扩展边界的自主性，决定了路线比较究竟应落在接口兼容、运行时组织还是交付治理层。
- (3) **峰值优化与工程闭环压力**：现有公开优化仍多聚焦于单算子或单模型的峰值性能，而相对忽视生产环境下的工程交付闭环。生产级推理系统的真实竞争力，更多体现为执行、状态与成本能否协同成立，而不是实验室环境下的指标罗列 [52]。

这些压力表明，单纯的“算子替换”已不足以支撑国产推理引擎持续演进。仅凭生态名录并不能充分解释不同路线的复杂度承接方式，因而后续分析需要进一步引入统一的比较尺度，用以连接执行路径、状态治理、成本协同与交付闭环之间的关系。

3 分析骨架与核心架构

从系统视角看，大模型推理引擎不是自下而上的模块堆叠，而是一套同时处理执行流、状态流与控制流的运行时。模型执行与算子层决定抽象模型如何映射到具体设备能力，缓存与调度层决定请求到达过程中资源如何被持续重写，而接口与运维层则决定这些内部状态能否以可升级、可观测、可诊断的方式进入真实生产。对国产平台而言，很多看似属于“局部性能”的问题，首先暴露在模块边界处：图模式回退是否需要被调度层感知，多模态前处理应停留在接口层还是进入状态系统，平台插件究竟只负责设备发现还是还要承担环境修复与后端选择。若这些边界定义不清，模型接入、性能优化与工程交付就会反复冲击共享主链。

因此，本文将国产推理引擎的核心架构压缩为“一个底座、三条主路径”。统一抽象、统一指标与可观测底座提供共同语义与证据链；执行与通信、状态治理、压缩与成本协同三条主路径分别回答系统如何运行、如何持有和迁移状态，以及如何在质量约束下控制成本；服务接口与交付闭环则检验这些能力能否稳定进入生产环境。表3 汇总了这一分析骨架下最核心的技术问题；若缺少其中任意一类，分析就容易停留在系统名录而不是系统机制上。

图1 以“服务入口-运行时核心-平台边界”三层压缩全文分析框架，强调国产推理引擎的性能、稳定性与交付能力来自层间联动，而不是某个热点内核的孤立优化。后文据此从最小推理闭环、统一分析底座、三条主路径和服务层收束几个层次展开，讨论国产平台上的复杂度如何被压缩到可治理边界内。

3.1 从最小推理闭环到统一分析底座

为便于后文讨论执行、调度和缓存问题，先用当前最常见的 decoder-only 自回归模型概括一次最小推理闭环。输入文本首先经过 tokenizer 切分并映射为 token id，再由 embedding 层转换为连续表示，并与位置信息一起形成 Transformer block 的输入。对本文而言，关键并不在于重复展开 Transformer 的教科书式结构，而在于说明这些计算单元如何被组织成可服务的运行时：多头注意力负责跨 token 建模依赖关系，前馈网络、归一化与残差连接共同完成逐 token 的表示更新。

对推理服务而言，prefill 与 decode 直接决定系统行为。Prefill 对整段提示词并行建模并建立 KV Cache；最后一个位置的 hidden state 经 LM head 和采样策略后产生第一

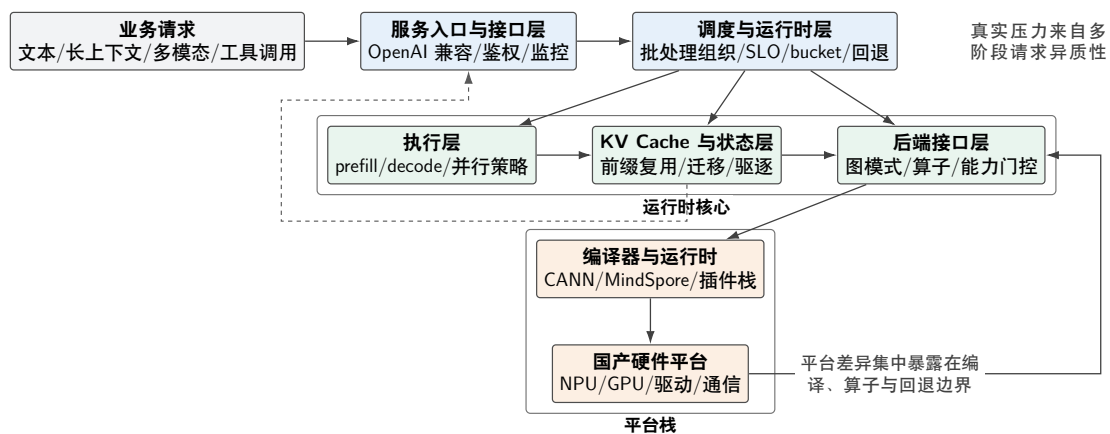


图 1: 国产推理引擎系统设计示意图

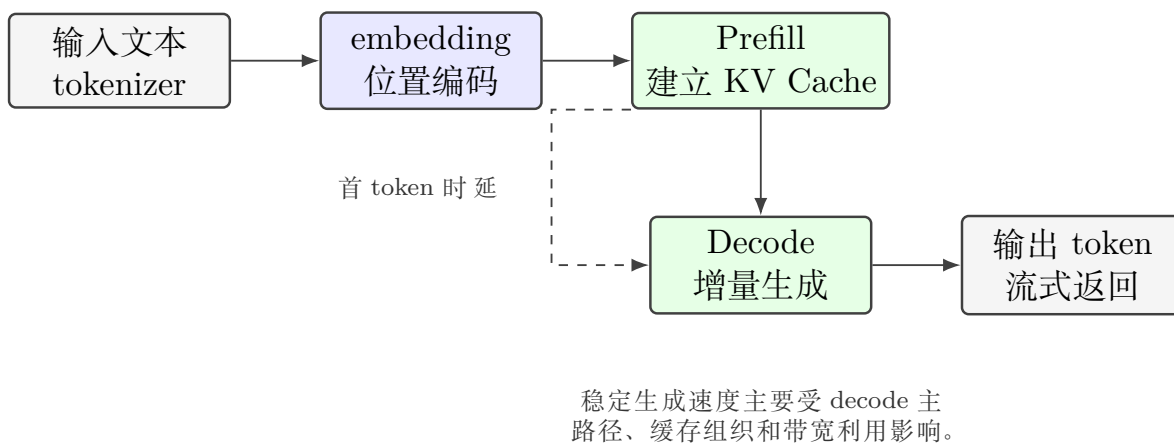


图 2: 从输入到输出的最小推理闭环

表 3: 国产推理引擎关键技术点总览

技术主路径	关键技术点	典型评价维度
执行与通信	统一执行 IR、动态图与图模式切换、Prefill/Decode 阶段解耦、推测解码协同、拓扑感知通信运行时、计算-通信重叠	MFU、吞吐、TTFT、TPOT、跨节点扩展效率、回退稳定性
状态治理	Prefix/共享状态索引、PagedAttention 与块化缓存、多级存储驻留、状态生命周期管理、跨节点状态迁移、命中感知调度	KV 命中率、迁移流量、恢复时延、显存/主存/NVMe 占用、长上下文稳定性
压缩与成本协同	混合精度量化、KV 动态量化、稀疏-量化协同、低比特算子路径、质量约束下的单位 token 成本优化	精度保持、吞吐/时延变化、状态容量变化、带宽占用、单位 token 成本
统一底座与交付	接口协议、指标口径、trace/log/metrics、回归验证、第三方测试证据链、环境治理与部署控制面	可复现性、可比较性、升级回归成本、交付成熟度

个输出 token；随后 decode 每步只处理新 token 的增量计算和缓存更新，直到满足停止条件。于是，首 token 时延主要受前处理、排队和 prefill 主路径影响，稳定生成速度则更多受 decode 主路径、带宽与缓存组织影响。连续批处理、chunked prefill 和阶段解耦，本质上都在利用前者更接近 compute-bound、后者更接近 memory-bound 这一差异 [4–7]。推理优化也会很快从“减少重复计算”转向“组织运行时状态”：KV Cache 通过保留历史 Key/Value 避免每一步重算整段上下文，PagedAttention、Prefix Cache、MQA/GQA、KV 压缩与跨实例迁移则进一步决定缓存如何被切块、复用、迁移和回收 [1, 53, 54]；与之并行，FlashAttention 侧重减少注意力计算中的高带宽内存访问，量化则通过降低权重或 KV 精度换取更低的显存与带宽压力 [55–58]。

近年来推理系统研究中的代表性工作呈现出相对一致的演进方向：早期系统更强调单一瓶颈的突破，例如连续批处理、PagedAttention 或注意力 kernel 优化；而随着 Agent、多模态、长上下文与结构化生成逐步进入主流负载，系统设计开始转向跨模块协同，把执行层、缓存层、调度层和约束解码视为一个统一运行时问题来处理 [1, 4, 8, 9, 55]。对于国产算力推理引擎而言，这一变化尤其关键，因为平台差异往往首先放大模块边界上的协同成本，而不是某个单点算子的理论性能损失。但仅有最小推理闭环仍不足以支撑路线比较。国产平台推理系统通常同时面对多类芯片、多套编译链、多种部署模式以及不断变化的模型版本；如果执行事件、状态对象、压缩配置和服务指标没有统一语义，那么即便局部优化已经实现，也很难在跨平台、跨阶段和跨任务条件下形成可比较的证据链。很多团队看似在做性能优化，实际却把大量时间消耗在“指标无法对齐、trace 无法归因、回归结果无法复现”的工程摩擦上。

因此，统一底座的意义在于为后续主路径优化提供一致的能力口径：执行层统一描述 prefill/decode 阶段、图模式切换和通信事件；状态层统一描述前缀命中、驻留层级、迁移开销和生命周期；压缩层统一描述量化配置、精度约束、状态容量变化和单位 token 成本；服务层则把 TTFT、TPOT、吞吐、尾时延和异常回退记录到相同口径中。在这一

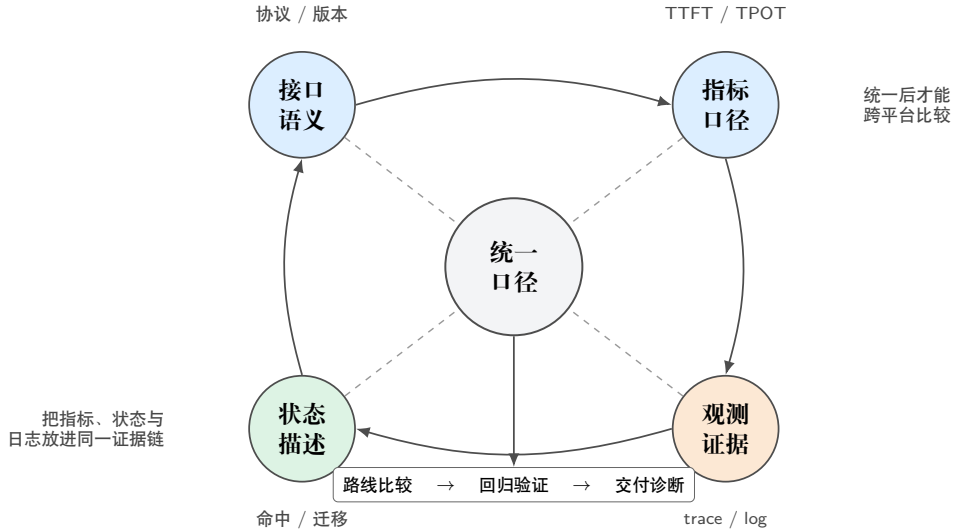


图 3: 统一抽象、指标与观测证据链示意图

前提下，不同平台、不同版本和不同优化策略的收益与代价才可能被稳定比较，第三方测试、阶段回归和工程交付也更容易形成可复用证据链。进一步地，高频指标还需要被压缩为可以复用的统一口径：哪些指标用于描述服务体验，哪些指标用于描述状态组织质量，哪些指标用于描述后端执行效率，哪些指标只能在特定负载边界内解释。相关口径被稳定定义后，不同平台、不同执行模式和不同压缩策略之间的比较才不易出现语义漂移。

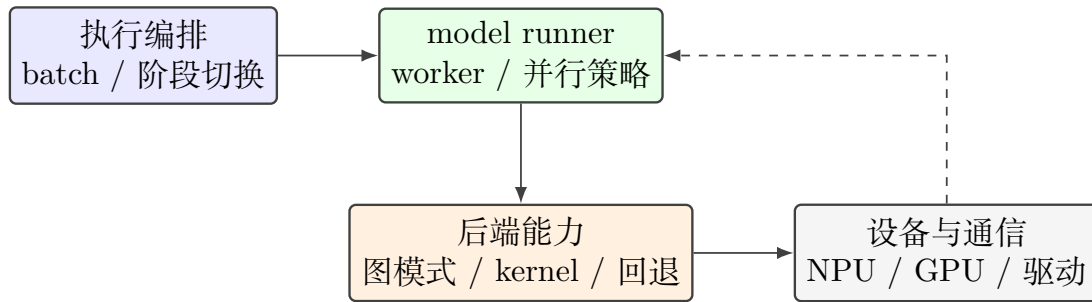
3.2 主路径一：执行编排、算子与后端能力边界

模型执行层负责组织 Prefill 与 Decode 两类核心路径，协调张量并行、流水并行、专家并行等分布式策略，并在运行时管理设备上下文、流与事件。对于推理系统而言，这一层不仅决定算子调用顺序，还决定请求合并粒度、静态图与动态图切换时机，以及多租户场景中的资源隔离方式。国产算力场景下，该层通常还承担设备能力探测、后端选择与降级回退职责。与成熟 CUDA 生态相比，国产硬件在图编译约束、动态 shape 支持、通信原语完备性以及自定义算子接入方式上差异较大，因此模型执行层往往需要显式管理更多平台相关信息，例如是否启用图模式、哪些路径必须回退到 eager 执行、哪些模型结构需要特殊 patch 才能稳定运行等。执行层因而更容易成为国产适配中产生侵入式分叉的区域。更可持续的组织方式，是将设备能力判断、图模式封装和模型特化逻辑尽量收敛到平台适配层或插件层，而不要直接散落在通用调度逻辑中。这样做虽然会增加抽象设计成本，却能显著降低后续跟踪上游版本时的合并压力。执行层既不能退化为纯粹的设备调用外壳，否则无法吸收模型结构变化；也不能无限吞并平台特例，否则又会成为分叉最严重的区域。让执行层显式依赖平台能力描述、缓存策略接口和调度回调，而不是直接携带某个平台或某类模型的特判实现，新增复杂度就更容易在后续扩展 MoE、长上下文、多模态和语音链路时被映射为局部能力增量，而不是整条执行主循环的持续膨胀。

从更广义的推理系统研究脉络看，执行层与运行时设计一直围绕吞吐、时延与资源受限条件之间的平衡展开。Pope 等对 Transformer 推理可扩展性给出了系统分析，FlexGen 展示了受限显存条件下的卸载与调度策略，StreamingLLM 与 LongServe 则进一步讨论了流式状态维持和长序列服务组织问题 [59–62]。这些研究虽然主要面向通用硬件环境，但

其关于运行时状态组织、带宽瓶颈和阶段切分的结论，对国产算力推理引擎同样具有直接参考价值。

执行层今天已经不再只是“执行模型前向计算”的封装层，而更像是协调多类状态生命周期的控制层。一方面，它要感知模型结构本身的变化，例如 MoE 路由、MLA 注意力、视觉编码分支或语音前端引入的新阶段；另一方面，它又要把这些变化映射为平台可接受的执行图、批处理形态和并行策略 [63–67]。如果国产平台上的运行时抽象仍停留在“设备适配 + 算子替换”层面，就很难支撑模型结构快速演化背景下的长期维护。执行层一旦真正落到国产平台，最先被收紧的往往不是调度策略，而是算子、图编译与后端接口这些能力边界。主路径一的难点不仅在于如何组织模型执行，还在于如何把平台特化加速收敛为可维护的后端契约。



平台差异更适合被封装在后端能力边界内，而不是持续回流到共享调度主链。

图 4：执行编排与后端能力边界示意图

算子与图编译层是国产推理引擎差异化最明显的部分。大模型推理涉及矩阵乘、归一化、注意力、采样、缓存搬运和多模态编码等多类操作，其中既有适合由厂商编译器或融合引擎统一优化的规律性计算，也有强依赖运行时状态的控制型逻辑。对于国产平台，系统设计者通常要在三种路径之间权衡：直接复用通用框架算子、通过图编译获得整图优化，或为热点路径实现定制算子。

这一层更值得观察的，并不是某个 kernel 在单一芯片上的峰值数字，而是系统是否能够把平台特化加速收敛为稳定的后端能力契约，并像图4 所示那样，把这种差异约束在共享执行主链之外。

这一层的关键不只是单点性能，还包括接口稳定性与维护成本。如果一个后端为了获得局部加速效果而在共享路径中嵌入大量平台条件分支，那么一旦上游框架修改模型执行接口、缓存布局或采样流程，本地后端就会同步承受较高维护负担。因此，面向国产算力的设计应优先使用平台注册表、设备抽象、能力门控和后端插件接口，把“平台差异”封装成可替换模块，而不是扩散成全局条件判断。近期围绕 NVIDIA Hopper 的工作把这种硬件耦合展示得尤为充分：FlashAttention-3 直接利用 Tensor Core 异步执行、TMA 与 FP8 支持提升 H100 上的注意力利用率，而 FlexAttention 则试图在保留 fused kernel 性能的同时恢复 attention 变体的可编程性 [57, 68]。这类工作说明，高性能推理后端的难点往往不在“有没有 kernel”，而在“如何把具体硬件代际能力组织成可持续复用的编译与运行时接口”。

因此，算子和图编译层更需要沉淀的并不是某几个孤立最优 kernel，而是一套更稳定

的后端能力契约：哪些算子变体可以被图模式安全覆盖，哪些路径必须保留 eager 回退，哪些低比特或结构化输出逻辑需要在运行时显式感知，哪些优化能够以插件形式随平台演进而单独升级。对国产平台而言，这种契约化表达尤为重要，因为很多问题并不是“后端不可用”，而是“某种能力只在某些约束下可用”。若系统无法表达这种带条件的能力边界，调度器和执行层就只能以越来越多的探测和分支维持稳定，最终把后端问题重新扩散回共享主干。

3.3 主路径二：状态治理、KV Cache 与长上下文组织

状态治理的核心载体是 KV Cache 及其相关元数据系统，它直接影响并发能力、会话恢复成本与长上下文性能。PagedAttention 一类设计通过块化管理显存，降低了连续大块分配的压力，也为前缀复用、分层缓存与按需换入换出提供了实现基础 [1]。在在线服务中，KV Cache 已不再只是“中间结果缓存”，而是决定系统并发上限、会话恢复成本和长上下文单位 token 成本的关键资源。对于国产硬件，状态系统还面临两个额外约束：其一，不同后端对内存分配粒度、数据搬移代价与 host-device 同步开销的敏感度不同，导致在 CUDA 上有效的缓存布局未必可以直接迁移；其二，当系统需要支持 Prefix Cache、Chunked Prefill、分层缓存乃至跨设备迁移时，缓存元数据管理复杂度会显著提升。此时，如果缓存系统与调度策略、图编译策略耦合过深，就容易在长上下文或高并发下出现资源碎片化、性能抖动和回收不及时等问题。相关研究与工程实践已经表明，长上下文场景下的缓存与调度协同是推理效率的决定因素之一。无论是 Orca 对连续批处理和迭代级调度的强调，还是 Sarathi-Serve 对 Chunked Prefill 的系统化组织，都说明内存与调度不能被割裂看待 [4, 5]。

进一步看，围绕注意力高效实现与缓存压缩已经形成一条清晰的技术主线。FlashAttention、FlashAttention-2 与 FlashAttention-3 从算子层逐步降低注意力计算的内存访问代价，并展示了 kernel 设计如何随着 GPU 代际演进而持续贴近具体硬件特性 [55–57]；MInference、DuoAttention、vAttention 与 LServe 则分别从动态稀疏注意力、检索头/流式头分离、虚拟内存式管理以及统一稀疏注意力执行角度改写了长上下文推理的系统组织方式 [69–72]；H2O、KIVI、SnapKV、PyramidInfer 与 CacheGen 等工作则分别从重点点击中、低比特量化、语义相关压缩、层次化压缩和缓存流式传输角度缓解了 KV Cache 的显存与带宽压力 [53, 54, 73–75]。这些工作共同说明，国产推理系统若要支持长上下文和高并发，必须同时关注注意力算子、缓存布局与压缩策略的协同设计。

因此，国产推理引擎在 KV Cache 设计上更适合采用“统一抽象 + 平台特化实现”的方式：统一暴露块管理、引用计数、前缀复用和驱逐策略接口，而将具体的数据排布、搬运实现和异步执行细节留给后端适配层处理。从系统演进角度看，KV Cache 还越来越像独立基础设施而非执行层附属模块。随着长上下文、多轮会话、多 Agent 工作流和多模态前缀共存，缓存不再只是服务某一次 decode，而是在更长时间尺度上决定哪些状态可以复用、哪些请求值得迁移、哪些阶段应被解耦执行。也就是说，缓存系统未来不仅要回答“数据放在哪”，还要回答“状态如何被计价、回收、复用和转移”。若缓存层长期停留在局部内存优化的定位上，后续很多关于长上下文和复杂负载的优化都会因状态治理能力不足而难以稳定落地。

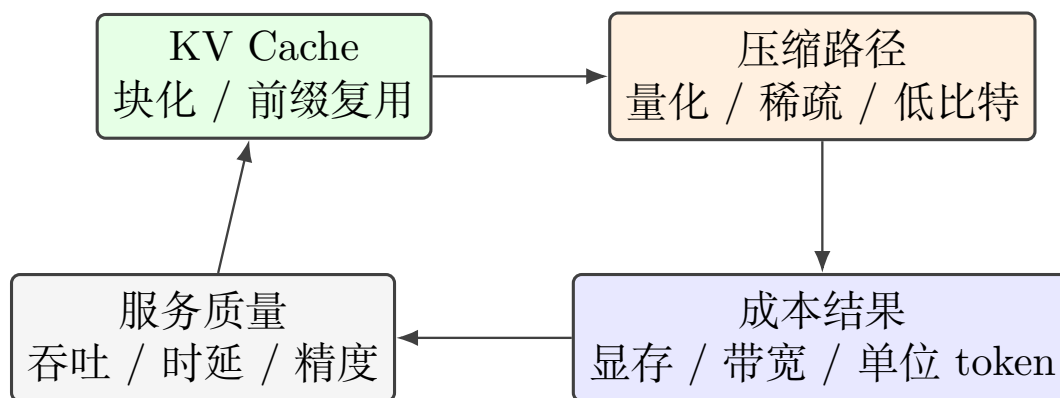


图 5: KV Cache、压缩与成本反馈闭环示意图

3.4 主路径三：压缩、成本与服务质量协同

在国产平台语境下，压缩加速不应再被看作部署后期附加的一组“省显存技巧”，而应被视为与执行路径和状态治理并列的核心主路径。一方面，混合精度量化、KV 动态量化和稀疏-量化协同确实能够直接降低显存占用、带宽压力和单位 token 成本；另一方面，它们的实际收益又高度依赖后端是否存在稳定低比特路径、状态层是否允许容量变化、调度层是否能吸收由量化带来的 batch 形态变化。也就是说，压缩配置的效果从来不是单独由模型精度决定，而是由“压缩参数、执行效率、状态占用、服务质量”四者共同决定。

如图5 所示，压缩路径的工程价值需要回到 KV 组织、服务质量和单位 token 成本的联动关系中才能被正确评价；脱离这一反馈闭环讨论量化或稀疏化，往往只能得到局部最优。

相关系统已经说明，压缩能力只有进入运行时闭环后才具有工程价值。QServe 通过把权重量化、KV 压缩与 serving 系统协同组织，展示了低比特部署为何必须在 kernel、调度和缓存之间联合设计 [58]；KIVI、SnapKV 等工作则进一步表明，长上下文场景下的 KV 压缩只有和缓存布局、热度判断及恢复路径结合起来，才可能在不显著破坏服务质量的前提下释放收益 [53, 54]。因此，国产推理引擎若要把“低成本”转化为系统能力，就必须让压缩配置、运行时观测与成本核算使用统一接口，而不是把量化与稀疏化停留在模型离线导出环节。在这一前提下，单位 token 成本才会从宣传指标变成可以稳定优化、回归和交付的工程指标。比较国产推理引擎的压缩路线时，也更应关注系统是否建立了可回归的成本与质量反馈机制，而不只是统计系统支持了多少种量化格式或低比特变体。

3.5 服务层收束：调度、接口与交付闭环

服务层需要在吞吐与时延之间平衡，典型机制包括连续批处理、请求优先级控制、SLA 感知调度、流式返回，以及结构化输出或工具调用时的状态管理。对于国产硬件，还需要考虑编译图缓存、静态形状约束与后端初始化成本。也就是说，服务调度不仅要优化“请求何时执行”，还要优化“请求如何映射到平台最擅长的执行形态”。在简单文本生成场景中，调度目标通常是通过更高的 batch 命中率来提升吞吐；而在复杂 Agent 场景中，系统还必须处理函数调用、结构化约束解码、多轮上下文恢复和外部工具返回后的再进入执行。这使得调度器需要持有更丰富的请求状态，同时与采样器、缓存管理器和运行时形成更紧密的协同。XGrammar 说明结构化生成已经不再只是前端语法约束，而是需

要与推理引擎共同设计、尽可能把额外开销压缩到接近零的系统问题 [8]；AI Metropolis 则进一步表明，当负载扩展到多 Agent 仿真时，依赖感知的乱序执行会成为提高并行度和硬件利用率的关键机制 [76]。对国产平台而言，如果底层后端对 shape 波动较敏感，那么调度器还需要通过 bucket 化、请求整形和预热机制减少编译抖动。

这一方向上的代表性路线还包括围绕高性能 serving 栈形成的部署体系。以 TensorRT-LLM、DeepSpeed-FastGen、FlashInfer 与 QServe 为代表的项目或系统，分别从图执行、运行时优化、高效算子以及低比特量化协同设计角度推动了推理系统的工程演进 [58, 77–79]。其中不少优化明显带有硬件特征，例如面向 NVIDIA 的 TensorRT-LLM 插件、Hopper 上的 FP8/异步 attention 路线，以及围绕 CUDA core 吞吐瓶颈展开的低比特 serving 设计。进一步看，TensorRT-LLM 已把多模态支持做成相对完整的运行时能力集合，包括独立的 multimodal runtime mode、模型支持矩阵、面向多 GPU 的“LLM tensor parallel + 视觉编码器复制”组织方式，以及在 chunked context 下的 embedding table offloading 等机制，这说明多模态部署已不再只是模型转换脚本问题，而是进入了缓存、并行与内存管理共同参与的运行时层 [80]。与此同时，OmniServe 开始尝试把 QServe 的低比特量化路径与 LServe 的长上下文统一稀疏注意力纳入同一套 GPU serving 框架，说明量化、高吞吐与长上下文优化正从孤立技巧走向组合式运行时设计 [81]；而在 AMD 生态中，围绕 MI300X 的工作则更多强调 prefix caching、chunked prefill、speculative decoding 以及 GPU partitioning 下的多实例部署权衡 [82–84]。这些工作虽然不直接针对国产算力，但为本土系统理解热点路径优化、调度协同、量化部署和工程化取舍提供了有价值的对照。

在调度策略方面，研究社区还提出了多种更激进的阶段解耦与服务组织方式。例如 DistServe 和 Splitwise 将 Prefill 与 Decode 的资源需求进一步拆解，以提升系统整体 goodput 或降低阶段间相互干扰 [6, 7]；Llumnix 通过跨实例动态迁移请求及其内存状态缓解异构请求导致的负载失衡，SOLA 则进一步把调度目标显式对齐到 TTFT 和 TPOT 等服务级目标，代表了从“吞吐优先”向“状态感知和 SLO 感知”调度的演进方向 [85, 86]。这类思路对国产平台尤其重要，因为图编译成本、shape 敏感性和设备初始化路径往往会放大不同阶段之间的资源竞争。因此，国产平台上的调度问题不能只被看作 batch 排队问题，而应被理解为一种“执行形态控制”问题。调度器需要决定的不只是谁先执行、谁后执行，还包括请求应被压缩进哪类 bucket、是否值得为某类形状维持图缓存、何时因工具调用或多模态阶段切换而暂时退出主循环、何时为了尾时延而主动放弃局部吞吐。若没有这种更强的执行形态意识，服务层即便短期内能提高平均吞吐，也很容易在真实业务负载下出现图缓存命中不稳、阶段争用严重和尾时延放大的问题。

状态治理继续向生产环境推进后，最终会落到交付闭环。推理引擎的对外层不仅包括 OpenAI 兼容接口、批量离线接口与嵌入式部署接口，也包括监控、日志、链路追踪和故障恢复等运维能力。这一层的重要性在国产推理系统中经常被低估，但在政企和行业部署中，系统往往首先因兼容性、可观测性或升级复杂度而被淘汰，而不是因为理论峰值性能略低。对外接口层的成熟还体现在生态兼容能力上。无论是以 Transformers 为核心的模型与 tokenizer 接口、以 Ray Serve 和 Triton Inference Server 为代表的服务编排与部署接口，还是以 FastChat 为代表的对话服务中间层，都在事实上塑造了现代推理引擎需要兼容的工程边界 [87–90]。因此，一个成熟的国产推理引擎架构应当把性能优化与可运维性一起视作一等能力。前者决定系统上限，后者决定系统能否稳定进入真实生产

环境。

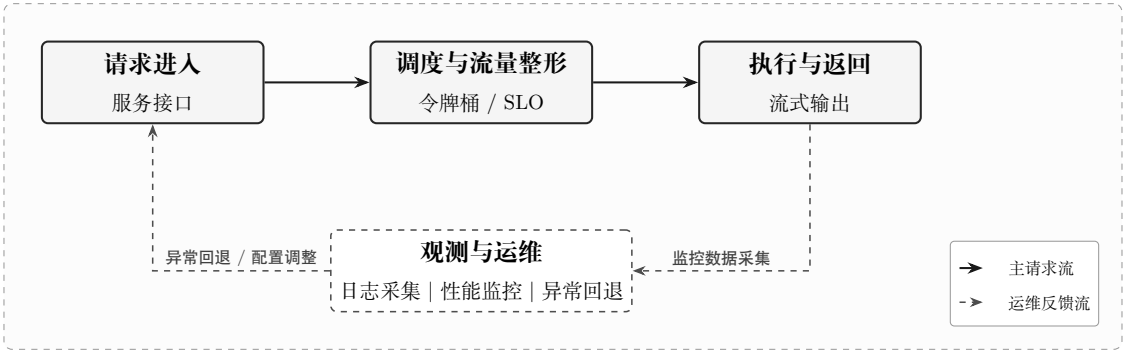


图 6: 调度、服务与运维闭环示意图

综合来看，国产推理引擎的架构难点并不分散在若干孤立模块里，而主要集中在模块交界处：执行层与图编译层决定可运行边界，缓存与调度层决定长上下文和并发表现，对外接口与运维层决定系统能否形成生产闭环。后文在讨论典型技术路线时，重点关注的也正是不同系统如何在这些边界上做取舍，而不是简单罗列其“支持了哪些功能”。因此，讨论国产推理引擎架构时，更值得比较的不是模块名单本身，而是这些模块之间是否形成了稳定的责任分层。一个系统即便短期内通过大量特判也能跑通模型，如果执行、缓存、调度与接口之间没有清晰边界，后续一旦遇到上游模型改动、平台版本切换或业务负载变化，维护成本就会迅速放大。相反，那些能够把平台能力、运行时组织和运维闭环收敛进统一分层的系统，即使局部优化未必总是最激进，也往往更有机会沉淀为可持续演进的工程底座。核心架构的优劣，最终不取决于某一层是否拥有最复杂的实现，而取决于系统能否把高变动能力隔离在可治理的边界内。对国产推理引擎而言，平台抽象、运行时状态管理和运维闭环不是三个可分离的附加维度，而是同一套架构是否成熟的不同侧面。不同技术路线的比较最终也会落到同一问题：何种架构组织方式更能在国产平台上同时维持性能、稳定性与可演进性。

4 基于分析骨架的路线比较

前两节已经分别梳理了国产推理生态的参与者及其分化动力，并建立了“一个底座、三条主路径”的统一分析骨架。以下将前文识别出的几类路线放到同一把分析尺子下比较，考察不同路线究竟把复杂度首先压在 backend、platform runtime、service state machine，还是 toolchain/control plane 上。

本章关注的不是“哪类路线先出现”或“哪类路线更像主流”，而是它们如何处理统一执行 IR、阶段解耦与通信运行时，如何组织前缀索引、多级驻留和状态迁移，以及是否把混合精度、KV 压缩和单位 token 成本带入同一优化闭环。这些差异决定了国产推理系统的真实上限，也决定了后续系统挑战首先会在哪一层暴露。

以下比较按复杂度主要落点展开：先讨论把复杂度压在执行主链与后端边界上的开源主干复用增强路线，再讨论把复杂度压在平台交付链和模板治理上的一体化方案，最后讨论把复杂度压在自研状态机、部署入口和本土能力沉淀上的国产原生与独立路线。

4.1 比较对象一：把复杂度压在执行主链的开源主干复用增强路线

开源主干复用增强路线通常将复杂度主要压在执行主链与后端边界附近，尽量继承上游框架已经成熟的调度、缓存、结构化输出和多模态能力，再把国产平台差异收敛为 plugin backend、worker、model runner、kernel 和外围治理工具 [2, 3, 22, 91]。这一做法的主要收益，是模型和 serving 特性的跟进速度较快；相应代价则是版本矩阵、异常回退和平台约束会持续反向挤压共享执行链，因此其优劣首先体现为边界经营能力，而不是单次性能收益。

这一路线的内部至少包含两种组织方向。第一种是 vLLM-Ascend、vLLM-MLU、SGLang Ascend 这类主线后端适配：它们尽量保持上层 OpenAI 兼容接口、调度语义与缓存抽象不变，再把 Ascend NPU、寒武纪 MLU 等平台差异压入 plugin backend、attention backend 与分布式运行时边界 [22-24]。第二种是 vLLM-HUST 这类本土增强实践：它不直接重写 serving 主体，而是在主干周围补齐平台护栏、环境预检、场景能力注册和 AGI4S 横切扩展，使平台特化与场景扩展尽量留在共享主链之外 [25]。前者更强调把国产后端纳入上游语义，后者更强调在不丢掉上游接口的前提下经营本地治理层；两者都属于“复用主干”，但复杂度压点并不完全相同。

按第 3 节的分析骨架看，这一路线在执行与通信主路径上最有优势，因为它能够最快继承连续批处理、chunked prefill、PD 分离、structured outputs 等主流机制；但在状态治理和压缩成本两条主路径上，它也最容易受到上游抽象边界的约束。平台侧若缺少稳定 kernel、graph mode、量化路径或分布式语义，本地团队往往只能在共享主链附近补丁式吸收这些缺口，从而把问题重新带回版本协同、运行模式选择和异常回退路径。也正因此，这一路线的证据边界必须写得更谨慎：主线纳入后端，并不等于所有高级能力已经稳定落地，roadmap、演示特性和可复现生产能力不能混写 [92]。

从系统分层看，这一路线更值得比较的不是项目名称，而是它如何分层：入口与启动护栏负责 CLI/OpenAI 服务和运行时预检，配置装配层负责把参数收敛为统一配置对象，引擎与 EngineCore 负责请求编排和调度通信，模型执行层负责 registry、worker、runner 与热点算子，平台与插件层负责能力探测和后端激活，多模态/Reasoning/Tool 层负责复杂能力的横切扩展，编译、KV Cache、分布式与 tracing 则构成性能基础设施 [25]。这一结构说明，开源主干复用增强路线并不是笼统地同时追求兼容与优化，而是通过分层把不同类型的国产化改动压回不同边界，使执行热路径优化、平台接入、环境治理和复杂能力扩展不必全部挤在同一条共享链路上。

从国产算力优化视角看，这种组件分层有助于识别不同组件的本土化优化重点。相比只给出系统级性能结论，表5 更细地展示了基于开源主干复用的增强路线各组件与国产算力潜在优化点之间的对应关系，包括 graph/compile 优化、插件化平台适配、多模态与工具调用状态机承载，以及运行时护栏与环境治理。vLLM-HUST 在这里对应的是这种组织方式的一个实例，而不是独立路线的代名词。

这一类生态已不再局限于少数头部系统，开源 serving 生态正沿几条较清晰的机制路径分化。LightLLM、Text Generation Inference 与 MLC-LLM 更强调面向在线服务的高吞吐批处理、跨设备部署和统一服务接口，代表“优先优化服务主路径”的路线；Ollama、llama.cpp、ExLlamaV2 与 mistral.rs 更突出轻量化部署、本地运行和低比特执行，代表“优先保障受限资源下部署稳定性”的路线；Aphrodite、KTransformers 与 Lorax 则更关

注运行时扩展、多租户适配和可定制 serving，代表“把推理引擎构造成可工程化组合平台”的路线 [93–102]。这一区分对国产算力同样重要，因为“兼容开源生态”并不等于接入某个框架，而是要明确优先继承哪一类能力主线：吞吐优先、受限资源部署优先，还是多租户与平台化能力优先。

进一步看，开源主干框架在不同硬件生态上的演进方式并不相同。NVIDIA 侧更快沉淀出围绕 Hopper、TensorRT-LLM、FlashAttention-3、FlexAttention 和 FlashInfer 的 kernel、编译优化与多模态运行时组合，特点是把硬件代际能力迅速转化为 fused kernel 与 runtime 特性；AMD 侧则更常围绕 MI300X、ROCm、vLLM 以及 SPX/DPX 分区模式组织多实例 serving、prefix caching 和前缀/解码阶段优化，重点偏向在现有框架边界内重组部署形态与资源切分 [57, 68, 77, 79, 82–84]。这些路线虽未必直接面向国产算力，却共同构成当前推理引擎生态的参照系，使国产后端适配必须同时回答两个问题：如何兼容主流接口，如何把本土平台特征转化为具体机制收益。

4.2 比较对象二：把复杂度压在平台交付链的一体化方案

平台一体化方案与前一类路线最大的区别，不在于是否使用了某个特定框架，而在于它把复杂度优先压进 platform runtime、compiler、服务模板和交付流程内部，以换取固定业务场景下更强的确定性。这里的比较重点不是“是否更强绑定厂商”，而是它能够把多少原本会暴露在执行主链上的波动，提前吸收到平台内部：图编译与 eager 回退的切换不再由业务侧逐请求决定，KV Cache 容量和批处理参数不再以分散配置存在，版本矩阵、镜像和监控模板也不再由使用者自己临场拼装。

在国产算力语境下，MindSpore、CANN 与 MindIE 构成了较典型的软硬一体化推理路径，百度飞桨/昆仑芯围绕 PaddlePaddle 与 FastDeploy 组织的原生部署栈也体现了类似思路 [14, 103–105]。这些对象之所以适合被放在同一比较面上，不是因为它们长得一样，而是因为它们都试图把执行形态、状态组织与交付模板一并收进平台栈内部，使性能上限更多来自“芯片特征 + 运行时 + 服务策略”的联动打磨，而不是单点替换某个 kernel [57, 82–84]。

这一路线的主要特征，体现为三类被平台内部吸收的波动：执行形态波动被模板化，状态空间波动被默认策略化，交付节奏波动被版本矩阵化。正因为这些波动被提前吸收，一体化路线在政企和行业部署中往往更有吸引力，因为系统是否能被稳定运维、统一回归和持续交付，常常比单轮模型跟进速度更重要。相应地，它的代价也很清楚：新模型、新服务特性和新任务形态通常要等待平台产品节奏释放，外部团队很难像使用开源主干那样快速做局部替换和组合式实验。

4.3 比较对象三：把复杂度压在边界经营的国产原生与独立路线

国产原生与独立路线的比较重点，不是“是否完全摆脱上游”，而是它是否主动把复杂度前移到 service state machine、KV Cache、阶段 DAG 和部署闭环这些更靠近运行时中心的位置。Chitu 与 xLLM 可以被放到这一类，并不是因为它们都自称独立引擎，而是因为它们都试图把多元算力适配、PD/EPD、长上下文、多级 KV Cache、容错治理和服务入口放进同一套运行时边界内 [16–20]。这意味着它们不是优先把复杂度压回后端插件，而是更早把复杂请求显式写成状态机。

因此，这一路线最大的收益，在于它更容易把文本、多模态、工具调用和多硬件部

署中的跨阶段关系直接纳入统一调度平面：哪些状态留在 service 层，哪些状态下沉到 engine 层，哪些资源竞争由 PD/EPD、KV 多级驻留或多流并行吸收，都可以在一套自治的边界内统一表达。对于国产硬件资源池而言，这一点尤其关键，因为硬件差异往往不是在单次 kernel 调用时暴露，而是在跨阶段迁移、异构设备协同和故障回退时集中暴露。如果系统已经把这些阶段关系写成显式状态机，多硬件兼容性就更可能被转化为可治理的调度问题，而不是不断回流为 backend 层面的局部补丁。

但这一路线的代价也最集中：一旦不再依赖上游主干吸收复杂度，系统就必须同时承担模型支持矩阵、服务接口、状态治理、部署入口和交付模板的长期维护压力。因此，评价国产原生/独立路线时，更应比较的不是“支持了多少能力”，而是这些能力是否已经沉淀为可持续演进的抽象，以及系统把生态迁移、平台适配、交付运维和复杂负载等成本压在了哪一层。DeepSeek-V4 可以被放在这一脉络中理解，也正因为其公开报告已开始把模型结构、专家并行、注意力实现、KV 基础设施与部署协同放进同一设计框架讨论 [21]。

从当前已经能够坐实的公开主体和产品形态出发，本土路线更适合按“复杂度主要落点”而不是按项目名划分：插件式后端适配把复杂度压在 backend 与上游接口边界，平台一体化把复杂度收进 SDK/runtime/compiler 与交付模板，国产独立引擎把复杂度前移到 service state machine、KV Cache 与部署闭环，工具链型部署把复杂度组织到模型接入、服务接口和硬件安装矩阵，跨架构编译/运行时探索则试图把重复适配成本压进 compiler/runtime abstraction [17, 20, 22–24, 105–108]。这样的分层能够更准确地对应各路线实际承担的复杂度类型。

公开材料中经常会同时出现工作站、一体机、云平台、训推平台和应用交付页面，但这些对象大多是在封装、承载或预集成上述引擎路线，而不是与推理引擎本体处于同一比较层级。部署载体回答的是能力通过什么形态交付出去，推理引擎回答的则是请求、状态、缓存、调度和算子如何被组织起来。若将两者直接并列，就容易把工程入口便利性误写成引擎内核先进性。

表6 所呈现的重点不是项目名录，而是国产算力推理引擎如何承载复杂度。复杂度主要落在 backend 时，路线优势通常是更快复用上游生态；主要落在 runtime 或平台交付链路时，固定平台上的确定性更强；主要落在 service state machine 和 KV Cache 治理时，更容易支撑动态 PD/EPD、长上下文和多硬件资源池；被压入 toolchain 或 compiler/runtime abstraction 时，则更有机会缓解多芯片重复适配与部署入口碎片化。面向国产算力的推理引擎竞争，因此更接近“谁掌握抽象边界、谁承担适配成本、谁负责交付闭环”的系统分工。

结合官方文档、源码入口和近期正式研究可以更清楚地看到，本土路线分化首先反映的是复杂度向何处外溢。插件式后端适配的摩擦主要体现在上游主干、插件分支、CANN/torch_npu 与厂商 SDK 的协同；平台一体化路线的压力更多落在服务模板、监控、配置基线与回滚路径；独立引擎和工具链路线则更容易把问题暴露在服务状态机、KV Cache、模型支持矩阵和硬件安装入口的一致性上 [19, 22, 105, 106, 109]。与此同时，近期正式研究对真实负载与资源受限 serving 的刻画也给出了与这些工程观察一致的证据：真实请求分布往往呈现长度重尾、前缀局部性波动和阶段不对称，因而系统摩擦通常先出现在批处理组织、缓存命中和控制面稳定性上，而不是先体现为峰值吞吐不足 [111–113]。这些材料的综述价值不是给出新的项目清单，而是说明国产平台的一线难点往往是先建

立稳定的系统组织，再谈局部极致优化。

由此看，开源主干复用增强路线与国产原生路线的核心竞争力，最终并不只看某个 benchmark 上的瞬时吞吐，而要看系统是否建立了“复杂请求进入后如何不失控”的边界秩序。一个独立引擎即使在局部 kernel 或单模型支持上暂时不如开源主干丰富，只要它能把服务状态、缓存生命周期、部署护栏和多硬件差异组织成相互配合的层次，就可能在真实业务负载中表现出更强的可持续性。反过来，如果一个系统只是把更多能力不断堆到入口脚本、配置模板或 backend patch 中，即便短期内功能覆盖看似完整，后续也很容易在模型升级、多模态扩展或跨机部署时暴露出边界塌陷问题。这也是为什么评价国产原生路线时，需要同时观察其服务状态机是否清晰、缓存治理是否前置、以及多硬件差异是否被压回受控接口，而不能只看“是否已经摆脱上游依赖”。

工具链型路线进一步说明，国产算力适配不只发生在单一 runtime 的后端层。FastDeploy 更接近飞桨生态中的多硬件推理部署工具链：它把 OpenAI/vLLM 风格服务接口、PD 分离、KV Cache 传输、量化、投机解码和 Chunked Prefill 组织在同一工程体系中，并面向昆仑芯、海光、Ascend、天数、燧原、沐曦等平台提供公开适配入口 [106, 114]。LMDeploy 则需要明确区分 TurboMind 与 PyTorchEngine：前者主要服务 CUDA 高性能路径，国产平台扩展主要通过 PyTorchEngine/Other Platforms 文档进入 Ascend、Cambricon 和 MACA 等后端，并且模型、设备与 eager/graph/量化模式的支持粒度并不对称 [107, 109, 115]。二者都更接近“部署工具链 + 服务接口 + 硬件适配层”的工程体系，而不是单一推理引擎。

LightLLM 代表了国内团队在高吞吐 serving、TokenAttention、Efficient Router、调度与内核生态上的重要补充，其公开文档中最明确的国产硬件线索主要是 MUSA 路径，默认主线仍明显偏 CUDA/Triton[93, 116, 117]。中国电信 Triton 统一跨架构推理框架则不处于 vLLM、LMDeploy 或 Chitu 同一层，而更像跨架构编译/运行时抽象：官方材料强调自研 Triton 跨架构编译器、统一算子库、vLLM-Triton 透明嵌入插件和图算融合编译器，用于缓解多国产芯片重复适配问题；但由于代码、许可证、版本矩阵和复现脚本尚未公开，相关迁移时间与性能损耗数字只能作为官方技术验证口径谨慎引用 [108, 110]。

从官方仓库和部署文档所呈现的安装流程、模型矩阵与服务组件边界，可以归纳出几个持续出现的工程事实：vLLM-Ascend 的工程可行性高度依赖“上游主干版本 + 插件分支 + CANN/torch_npu”严格对齐，目标是在尽量不改上层接口的前提下完成国产后端迁移 [22]；MindIE 的吸引力并不只来自单点推理性能，而在于它把镜像、服务化组件、监控和配置模板一起交付，从而把一部分系统复杂度前移到平台栈内部 [105]；FastDeploy、LMDeploy 与 SGLang 的公开资料都表明，PD 分离、分布式连通性、运行模式选择和模型支持粒度会直接影响实际可交付性 [24, 106, 109]。这些公开材料持续暴露的，不是某条路线的单点性能差异，而是部署不确定性由谁承担、以什么方式被吸收。

将这些公开技术信号继续收束，可以看到不同代表对象往往会在不同边界更早暴露工程前提、系统收益与维护负担。这里不再把材料写成直接的处方式结论，而是把官方文档、源码入口和公开部署说明中能够较稳定坐实的对象画像并列展示，用以说明复杂度分别首先落在 backend、runtime、service state machine 还是 toolchain 上 [17, 18, 22, 24, 105, 106, 109]。

表7 展示的不是对不同团队的直接处方式建议，而是公开资料下各代表对象更早暴露的工程前提、系统收益与复杂度落点。与后文表12 对照时，更应看到的是不同路线把

复杂度分配到不同系统边界：有的路线更早在版本矩阵和平台适配上显化，有的路线更早在平台交付与模板治理上显化，有的路线更早在复杂控制流和运行时状态空间上显化，还有的路线更早在模块边界与长期 merge-safe 演进上显化。

如果再把视角从“项目或路线”切到“负载类型”，不同技术路线的差异会被放大到不同主路径上。公开案例反复表明，短文本高并发、长上下文知识问答、多模态文档理解和工具调用 Agent 等负载，并不是把同一条执行链简单放大，而是分别把版本矩阵、缓存迁移、阶段异质性与复杂控制流等问题提前推到前台。表8 将这一关系进一步压缩为工作负载与路线承载重心的对应图，其目的不是给出固定配方，而是说明何种系统矛盾更常在哪些路线下首先显化。

围绕多租户与定制化模型服务的能力也在持续拉开系统差距。Punica 与 S-LoRA 表明，当 LoRA 适配器数量迅速增长时，系统需要在缓存复用、批内共享和运行时隔离之间重新平衡 [120, 121]。与此同时，Guidance、Outlines 与 XGrammar 等工具说明，结构化输出与受约束生成正在把“推理引擎”从纯粹的 token 生成器转变为可编排的执行系统；而 XGrammar 的论文实现则进一步表明，这种能力已经需要以语法执行引擎、持久化栈和 GPU 重叠执行等系统机制来支撑 [8, 122–124]。

调度能力同样正在成为新的分化点，但这里更重要的也不是系统名本身，而是它们分别把哪一类调度变量显式化了。Llumnix 把跨实例动态迁移和内存状态搬运带入运行时，使“负载均衡”不再只是请求排队问题；SOLA 把 TTFT、TPOT 等服务级目标直接写进调度目标函数，使“调度最优”从吞吐优先转向 SLO 优先；AI Metropolis 则进一步把多 Agent 依赖关系显式化，通过乱序执行削减伪串行等待 [76, 85, 86]。这一组工作共同说明，现代推理系统的竞争焦点已经从“是否支持连续批处理”进一步演进到“是否能够在复杂状态空间里稳定组织执行”，而这正是国产推理引擎在复杂业务负载下最不能回避的技术点。

图7 将主要路线放到同一张图里比较，其重点不在功能罗列，而在它们对“生态兼容性、平台特化深度、交付稳定性”三者的不同取舍 [25]。从课题化建设视角看，这种分化本质上也是对执行与通信、状态治理和压缩协同三条主路径控制权的不同分配，因此更能解释为什么一些系统在 benchmark 上接近，却在长期维护和场景扩展上差异巨大。

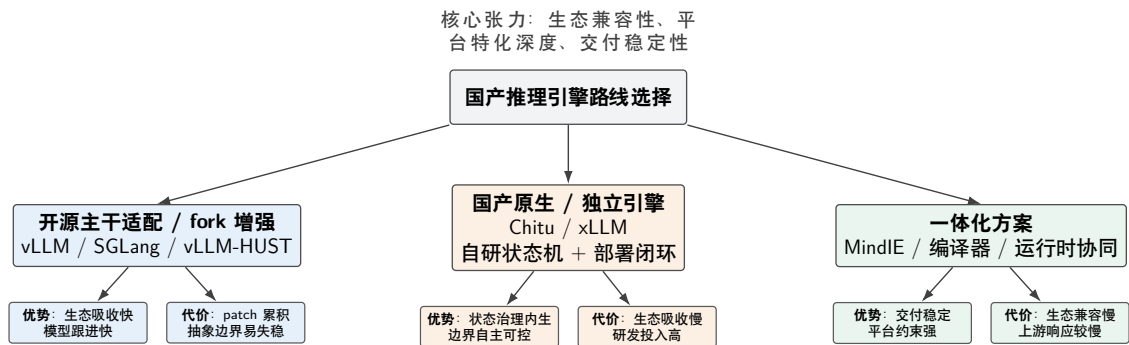


图 7：国产推理引擎技术路线分化示意图

就现阶段公开材料而言，基于上游主干的复用增强路线拥有相对更充分的证据支撑，因为它试图同时保留上游生态复用与本土关键路径特化。但这一判断成立的前提，是上层生态节奏与底层平台节奏之间存在清晰边界；否则系统很容易演化为既继承上游复杂度、又承受本地特化负担的双重分叉。

这些比较判断并非静态结论，而会随着上游版本迭代、国产后端接入深度和具体部署目标持续变化。因而更合适的表述，不是预设固定优劣顺序，而是比较不同代表对象把工程投入首先压在何处，以及公开证据对这种复杂度分配已支持到什么程度。后面的案例表也据此转向比较不同代表对象把工程投入压在何处。

多模态推理正在把引擎边界从文本 decode 推向异构阶段编排。就国内团队的工程案例看，Qwen2-VL 与 InternVL2 代表了多图与文档理解链路进入在线服务后的典型压力点：系统不仅要处理动态分辨率切图和图像 token 打包，还要面对视觉前缀复用不足、OCR 密集区域导致的 prefill 膨胀，以及图文混排输入带来的 shape 波动 [65, 66, 126, 127]。CogVLM2 则把视频理解显式纳入主流模型服务，使推理链路必须同时考虑帧采样、跨帧压缩、视觉 encoder 批处理以及 decode 侧连续 batching 之间的资源再分配 [128, 129]。在语音方向，Qwen2-Audio、GLM-4-Voice 与 MiniCPM-o 进一步把音频前端、speech tokenizer、流式 chunk 累积以及语音/文本交错生成纳入同一运行时，使首 token 时延、实时率和打断恢复直接成为推理引擎的系统约束 [67, 130, 131]。

从路线比较的角度看，这组现象进一步放大了三类路线之间的差异。多图文档理解首先冲击 prefill 主路径和前缀复用，视频理解首先冲击 encode/prefill/decode 的跨阶段再平衡，语音交互则直接改写会话状态机；因此，多模态路径扩张的并不只是“模型输入类型”，而是引擎内部必须长期维护的状态对象种类。公开材料显示，主干复用增强路线更常通过继承模型与 serving 特性来吸收这些变化；若视觉前缀、结构化约束和工具状态不能进入统一生命周期管理，相关复杂度就容易重新回流到共享主链。一体化路线更多通过固定负载下的 shape、graph 与部署模板收敛来压平波动，但新能力释放通常也更受平台节奏约束。国产原生/独立路线则更常把异构阶段直接组织为统一状态机，同时也相应承担模型矩阵、服务接口与部署闭环的长期维护代价。

ElasticMM、EPDServe 与 Eevee 分别从弹性并行、阶段解耦和模块复用三条路径回答这些问题：前者统一管理多模态请求与前缀缓存，后两者分别通过 encode/prefill/decode 拆分和 module multiplexing 缓解阶段异质性与模块争用 [9, 132, 133]。它们共同说明，多模态系统的难点已不是“支持某个模型”，而是能否在请求异构、阶段异质和模块干扰并存时保持资源利用率与首 token 时延稳定。对国产推理引擎而言，这要求把前处理、跨模态缓存、阶段解耦和异构编码链路一起纳入运行时设计。

这些工作在本章中的意义，不在于再补一轮多模态名录，而在于说明多模态、结构化输出和工具调用会把路线比较重新拉回“谁能以更低工程代价吸收模型变化”这一核心问题。无论是 LMDeploy 围绕部署工具链与模型支持矩阵的持续扩展，还是 MindIE 一类一体化系统对平台统一性交付的强调，抑或 vLLM-Ascend 与 vLLM-HUST 这类对象对上游抽象边界和本地治理层的不同依赖，其共同约束都不再只是“文本生成是否足够快”，而是多模态、结构化输出、长上下文和复杂工作负载能否被纳入同一套可维护的运行时框架 [22, 25, 105, 115]。

综合现有公开材料，可以得到几点较稳妥的观察：路线差异首先体现为复杂度被压入 backend、platform runtime、service state machine 还是 toolchain，而不在项目名录本身；不同路线对模型快速演进、固定平台交付和多硬件资源池的响应方式并不相同，这种差异最终会体现为版本矩阵、状态治理和交付闭环的不同压力分布；前文涉及的多模态、结构化输出和复杂控制流并没有改变比较框架本身，而是在持续放大这些复杂度分配差异。第五章所讨论的关键挑战，也正是这些差异在执行、状态、成本与控制面上进一步放

大的结果。

5 由路线比较收束出的关键挑战

第 4 节已经比较了不同路线分别把复杂度压在何处，这里进一步把这些比较结果收束为更稳定的系统挑战，考察复杂度被压在不同边界后最终会以什么形式重新回到系统主链。

综合前文，国产推理引擎面临的难点并不只是单点性能不足，而是多个系统目标同时受限：既要持续吸收上游能力演进，又要适应本土硬件和基础软件栈的边界，还要在真实业务负载下维持稳定交付。这些挑战之所以顽固，不是因为每一项都新，而是因为它们会相互放大。平台差异会抬高抽象成本，抽象不稳又会削弱复杂任务支持；复杂任务支持一旦采用侵入式补丁实现，又会加速本地分叉累积；而如果评测仍主要停留在理想化 benchmark，团队就很难及时识别这些结构性问题。表11 汇总了这种放大关系。这些挑战沿主路径扩散的方式并不相同：统一执行抽象、通信运行时和阶段解耦机制一旦缺位，系统通常会先在吞吐与时延主路径上失稳；前缀索引、多级驻留和状态迁移能力一旦不足，问题通常会先在长上下文和高并发条件下显现；量化、KV 压缩和成本核算一旦没有进入同一闭环，压缩收益就往往只存在于离线实验而无法沉淀为真实生产能力。下文按主路径和底座边界展开。

公开技术资料显示，一线团队真正要处理的问题与本文的挑战分解高度同构。官方仓库、部署文档和模型支持矩阵反复指向版本矩阵对齐、PD 分离与分布式稳定性、MindIE 服务化配置可控性，以及 LMDeploy 在国产 GPU/信创环境中的适配取舍 [22, 105, 109]。这些材料虽不等价于完整生产回放，但与近期正式研究对真实工作负载的刻画相互印证：生产级 LLM 服务的请求长度、前缀复用、阶段切换和突发模式都具有明显波动，因而真实摩擦往往先出现在环境治理、服务模板和平台工具链，而非单点跑分 [111, 113]。

从官方部署文档与公开仓库也能看出，真实部署约束往往围绕组织能力边界而非理论峰值展开。vLLM-Ascend 的适配资料将插件、环境、通信与运行模式检查前置，说明在信创或一体机场景里，“先稳定启动起来”本身就是核心工程门槛 [22]；MindIE、LMDeploy 与 SGLang 的公开资料则分别把平台交付、模型支持矩阵、执行模式与运行时接口放在显著位置，说明国产路线的真正差异常常落在系统组织方式而不是单点峰值 [24, 105, 109]。这些公开摩擦最终都可回收到几类更稳定的系统挑战：版本矩阵与插件对齐放大了平台抽象和共享路径分叉风险；PD 分离、双机组网和连通性放大了调度、状态迁移与网络控制面的耦合；模板与配置敏感性则说明一体化路线的真正门槛不只在执行性能，还在服务层参数、监控与回退路径是否足够可控。

结合前文的路线比较与本节的挑战分解可以看到，不同路线并不是面对同一组问题再做不同优化，而是会把“版本矩阵、网络控制面、缓存/调度、运维回退”这些挑战优先暴露在不同边界上。路线差异本身已经在重新安排系统能力首先承压的位置。表12 将这一关系压缩为更直接的工程归纳。

与第 3 节表7 对照时，关键结论不是“哪条路线天然更优”，而是“不同路线把复杂度分配到不同系统边界”：vLLM-Ascend 更早暴露版本矩阵与分布式连通性，MindIE 更早暴露模板治理与平台回滚，LMDeploy 更早暴露跨平台基线与量化回退，SGLang 更早暴露复杂控制流与执行状态组织，而以 vLLM-HUST 为代表的开源主干复用增强实践更早暴露模块边界经营与长期演进秩序。第 4 节真正要说明的，也正是这种“挑战先在哪

一层爆出来”的差别。

从问题诊断继续延伸到后续投入方向时，更工程化的问题就转化为：这些挑战分别可以通过哪些优化抓手缓解，又有哪些课题可以作为后续持续攻关的对象。表13 将执行、状态、成本、演进秩序和评测闭环对应到更清晰的优化与研究坐标中。

5.1 执行与通信主路径：硬件异构与后端碎片化

不同国产加速平台在编程接口、图模式约束、算子能力与通信栈成熟度上差异明显，导致共享推理路径中容易出现大量平台特化分支。若缺乏清晰的后端边界，系统会迅速失去可维护性。表面上看，这只是“多写几段条件分支”的问题，实际上它会逐步侵蚀调度、缓存、采样乃至接口层的独立演进能力。更棘手的是，硬件异构通常并不只表现为性能差异，还表现为正确性与稳定性边界不同。例如，某些后端对动态 shape 更敏感，某些后端对特定融合算子支持不完备，某些平台则在通信或异步执行语义上与主流实现存在差异。一旦这些差异没有被收敛到统一的后端抽象中，系统开发者就会被迫在共享路径上叠加越来越多的特判逻辑，使代码难以测试、难以合并，也难以为新模型复用。难点不在“是否存在统一抽象”，而在抽象该把什么纳入、把什么留在边界外。若抽象过薄，平台差异会反复溢出到 worker、调度器、采样器和 API 层；若抽象过厚，又容易把短期平台特例固化成长期接口负担。较健康的工程策略，通常不是试图用单一层吞掉所有差异，而是把平台能力差异、启动期环境治理和运行期异常护栏分层处理：平台层负责设备能力与后端选择，运行时层负责降级、回退和状态一致性，环境治理层负责依赖、驱动和工具链的前置修复。这种分层更重要的价值，在于能把不同类型的问题压回各自的责任边界，使系统在面对新硬件或新模型时不至于整条执行链一起失稳。

这也是为什么许多通用部署项目即便能够在共享生态中快速演进，一旦迁移到新的硬件后端，仍然需要经历较长的抽象重构周期。Text Generation Inference、MLC-LLM、llama.cpp、ExLlamaV2 与 Triton Inference Server 等系统对运行时、设备接口和算子组织做了不同取舍 [89, 94, 95, 97, 98]；而 vAttention、QServe 与 FlashMLA 这类较新的工作又进一步表明，一旦目标转向长上下文、高吞吐或低比特量化，系统边界和底层内核实现会比传统框架式适配更快成为瓶颈 [58, 71, 134]。因此，国产后端若要在这些生态上实现高质量适配，往往首先要面对的不是单点算子问题，而是系统边界和接口契约问题。

更具体地说，国产算力适配的工程挑战首先表现为版本矩阵而非单一后端接口问题：CANN、torch_npu、厂商 SDK、Ray、容器镜像、分布式通信库与上游框架版本之间往往存在强耦合，vLLM-Ascend、vLLM-MLU 与 SGLang Ascend 的公开材料都体现出这种依赖 [22–24]。其次，不同硬件上的 feature parity 很难默认成立，graph mode、attention backend、量化、PD 分离、KV Cache 传输和 speculative decoding 等能力即使在同一工具链中也可能存在支持粒度差异，FastDeploy 与 LMDeploy 的多硬件文档都需要按设备和执行模式分别核验 [106, 109]。再次，公开性能数字不能直接横向比较，因为模型版本、输入/输出长度、并发度、精度、batch 组织、硬件代际和 runtime 版本的任一变化都可能改变结论；因此 roadmap、官方宣传、技术验证和开源可复现能力应分层表述，尤其不能把 SGLang Ascend 的 roadmap 能力或中国电信 Triton 的技术验证口径写成已经完全成熟、可复现的开源引擎能力 [92, 108, 110]。

沿这一主路径，后续优化不能只理解为“继续补平台适配”，而应理解为把后端差异重新组织成更稳定的系统对象。相关演进通常表现为三项相互关联的建设方向：能力声

明前移，即把算子可用性、graph 约束、通信后端、量化支持和异常回退条件，尽可能收敛为运行前即可检查的能力契约；混合执行策略成形，即不再把 graph mode、eager mode、通信策略和低比特路径当作彼此独立的优化开关，而是把它们视为一组需要联合决策的执行选项；版本矩阵、feature parity 与分布式连通性转入持续回归对象，而不是每次迁移或升级时重新依赖人工排查。在这一框架下，后端优化的成熟标志，不再只是某次跑通某个模型，而是平台能力、执行模式和验证证据之间形成稳定闭环。

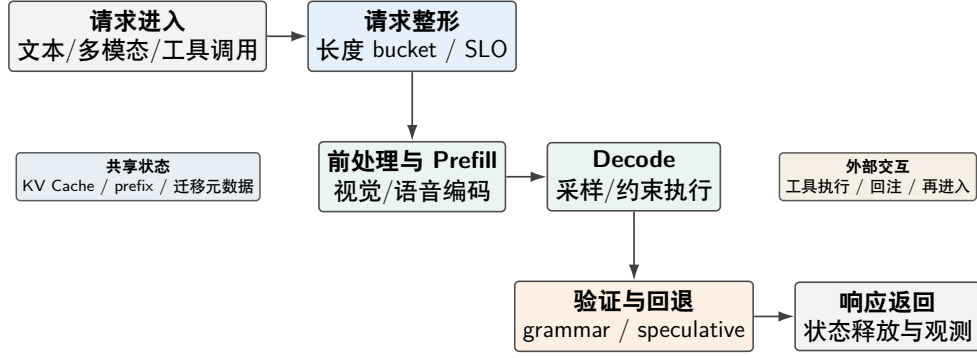
若把这一方向继续上升为课题，更值得深入的不是某一类平台私有算子，而是国产后端上通用执行边界如何被重新定义。例如，条件化能力 IR 需要回答的并不只是“后端支持什么”，而是“哪些执行假设在不同平台上仍然成立”；图安全回退语义要解决的也不只是异常时如何退回 eager，而是如何保证回退前后缓存、批次形状和分布式状态不会失配；跨平台 collective 规划则进一步把执行问题推向通信与拓扑层，要求系统能够根据互连结构和阶段特征选择通信组织方式。这些问题一旦被抽象清楚，平台适配就很容易长期停留在 patch 累积层面。相反，能力声明、混合执行和持续回归三者一旦打通，国产后端与主流共享生态之间的系统距离就更有可能被真正缩短。

5.2 状态治理主路径：复杂任务场景的能力补齐

当前真实负载已从单轮文本生成扩展到长上下文、多模态理解、结构化输出、推理链、工具调用和多 Agent 协同 [8, 9, 76]。相应地，推理引擎的优化目标也不再是单一路径 decode，而是跨阶段状态管理：结构化输出要求把约束执行压进采样环路，工具调用要求支持生成、外部执行与回注之间的往返切换，多模态请求要求消化更长的前处理链和更剧烈的 shape 波动，多 Agent 负载则要求识别真实依赖并消除伪串行。Parrot 进一步表明，当应用由多个 LLM 请求组成 workflow 时，如果服务系统仍停留在 request-level API，跨请求的变量依赖和数据流语义就无法进入运行时，端到端优化空间也会被明显压缩 [135]。对国产平台，难点不在接口名义上“可用”，而在这些能力会同时冲击调度、缓存和图模式。以国内多模态链路为例，多图与文档请求受视觉 token 膨胀和分辨率离散化约束，视频请求会放大 encode 与 decode 的资源竞争，语音对话则引入 streaming chunk、语音 token 和实时打断恢复；若运行时仍按文本模型的单阶段假设组织执行，就容易出现局部功能可用而端到端时延和稳定性失控的情况 [65–67, 129–131]。更关键的是，复杂任务场景直接改写了“状态”的性质。文本 serving 时代，系统重点管理的是请求队列、KV Cache 和 batch 形状；而进入多模态、工具调用和 reasoning 场景后，运行时还必须额外管理视觉前缀、语法状态、工具回注上下文、外部执行等待、流式中断与恢复等状态对象。它们既不是纯粹的模型内部变量，也不是可以完全交给服务层处理的外围数据，而是会直接影响调度顺序、缓存复用和回退路径选择的运行时实体。复杂任务支持的核心，不只是完成能力接入，而是要让这些新增状态在 prefill、decode、验证、外部交互与回收之间有明确且低成本的迁移规则。

图8 按时间顺序压缩了复杂请求进入运行时后的生命周期。国内常见的多模态、结构化输出和工具调用请求，会在前处理、约束校验、外部回注和状态回收之间反复切换；国产推理引擎的难点也随之落在多个阶段之间如何维持稳定的状态传递和资源复用。

下面这段概念性伪代码将国产推理引擎在复杂请求下反复处理的几个关键决策显式化：请求先被分类和整形，再决定是走 prefill、decode 还是多模态前处理分支；随后运行时还要同步维护图模式、缓存状态、结构化约束以及异常回退路径。复杂任务支持因此体



复杂请求的关键难点不在单步执行，而在跨阶段状态传递、资源复用与回退一致性。

图 8: 复杂请求生命周期示意图

现为一个贯穿执行、调度与缓存的闭环，而不是附着在文本生成主路径外侧的附加功能。

算法 1 复杂请求下的国产推理引擎运行时闭环（概念性伪代码）

```

1: Input: request stream  $R$ , platform capability set  $P$ 
2: initialize scheduler state, cache state, and backend registry
3: for all incoming request  $r \in R$  do
4:   classify  $r$  as text, long-context, multimodal, or tool-calling workload
5:   normalize prompt shape, length bucket, and service-level objective
6:   if  $r$  contains visual or audio prefix then
7:     run preprocessor and stage encoder outputs into shared runtime state
8:   end if
9:   if structured output or tool invocation is required then
10:    attach grammar state or external-call state to decoding context
11:   end if
12:   choose execution mode from eager, graph, or fallback path according to  $P$ 
13:   update prefix cache, KV blocks, and migration metadata before execution
14:   while  $r$  is not finished do
15:     schedule prefill, decode, or verification step under current batch plan
16:     monitor TTFT/TPOT pressure, shape drift, and backend exceptions
17:     if constraint violation or backend instability is detected then
18:       trigger rollback, degrade execution mode, or rebalance request placement
19:     end if
20:   end while
21:   emit response, release cache blocks, and record observability signals
22: end for
  
```

推测解码、约束解码和复杂控制流生成进一步抬高了对执行一致性和调度灵活性的要求。SpecInfer 已经说明，新的生成加速机制会引入新的验证与回退路径，如果后端抽象和服务编排没有预留空间，局部优化反而会放大系统复杂度 [136]。Leviathan、Medusa、EAGLE 与 Lookahead Decoding 分别代表了辅助模型、多头预测、特征不确定性建模与并行前瞻等路线 [137–140]。这些方法的共同前提是运行时必须支持更细粒度的候选组织、验证与回退。硬件侧经验也说明收益高度条件化：在 MI300X 上，speculative decoding 主要受益于小 batch、带宽受限的 decode，随着 batch 增大收益会明显收缩；进入多模态 serving 后，text-only drafter 与 multimodal drafter 还要在额外开销和 acceptance length

之间重新权衡 [141, 142]。因此，推测解码不是可独立开启的局部优化，而是与 batch 组织、图执行模式、视觉 token 压缩和验证链路深度耦合的系统机制。这类现象对国产平台尤其关键，因为本地硬件往往在图模式切换、动态 shape 支持和异步执行语义上更敏感。一个在主流 GPU 平台上“只是多一次验证”的机制，迁移到新的后端后，可能同时意味着图缓存失效、batch 形状离散化加剧和异常回退链变长。于是，复杂任务支持是否可靠，最终评估的就不再只是某种 decoding 技术本身，而是系统能否在能力增强后仍维持可预测的状态空间规模。真正成熟的国产推理引擎，应当把这些新能力视作运行时状态机的自然扩展，而不是临时外挂在主循环外侧的特性开关。

因此，状态治理主路径需要加厚的，并不是继续罗列复杂能力，而是明确这些能力在运行时里究竟以什么状态形式存在、如何流动、由谁回收。相应的系统演进自然落在三项相互关联的建设方向上：请求状态机显式化，使多模态前处理、结构化约束、工具回注、候选验证和外部等待状态进入统一调度视野；状态对象与资源对象联动，使视觉前缀、语法状态、KV 块和迁移元数据在同一运行时里共同参与 prefill、decode、回退和回收决策；复杂能力接入从“功能是否可用”推进到“状态迁移是否低成本且可预测”，因为真正决定系统上限的，往往不是某个 parser 能否工作，而是请求在 encode、decode、外部执行和中断恢复之间切换时，缓存命中、批次组织和异常语义是否还能保持稳定。

从课题方向看，这一主路径最有价值的研究对象，是状态如何成为推理运行时的一等公民。所谓状态中心化运行时，并不只是增加一层调度封装，而是要求系统能够用统一接口描述视觉前缀、工具上下文、结构化约束和 speculative decoding 候选，从而让这些对象共享同一套迁移、计价和回收规则。由此延伸出的关键问题包括：多模态缓存是否应与文本 Prefix Cache 共享统一命中逻辑，工具调用状态如何与生成状态协同进入调度器，PD/EPD 与阶段 DAG 调度在异构资源条件下怎样避免状态爆炸，以及多 Agent 控制流能否被压缩为可预测的状态转换图。谁能先把这些问题处理成运行时内部秩序，而不是外围特性堆叠，谁就更可能在复杂负载下真正建立护城河。

5.3 压缩与成本主路径：压缩加速与单位 token 成本协同

对国产推理引擎而言，“低成本”并不是部署阶段最后再去追问的经济性指标，而是与执行效率和状态治理同级的系统目标。问题在于，压缩策略的收益往往不是线性兑现的：同一组量化参数在不同后端、不同上下文长度和不同负载强度下，可能分别表现为吞吐提升、尾时延恶化、状态迁移变多或精度明显波动。于是，很多团队虽然在模型层面已经得到不错的压缩结果，但一进入真实 serving 场景，就会发现低比特路径与通信粒度、图模式命中、KV 驻留层次和调度桶化同时发生耦合，使单位 token 成本难以稳定下降。

压缩问题在国产场景下更容易从“模型压缩”演变为“系统协同”问题。QServe 说明，只有把量化、kernel 组织和 serving runtime 一起设计，低比特收益才可能在真实服务路径中落地 [58]；KIVI、SnapKV 等工作则进一步表明，长上下文场景下的状态压缩如果不和缓存元数据、恢复路径与服务质量门限一起考虑，就会把容量节省重新转化为恢复时延或命中率损失 [53, 54]。进一步地，Prism 表明，当服务目标从单模型优化扩展到多模型共池时，单位成本能否下降还取决于跨模型显存协调和 GPU sharing 策略能否随运行时需求动态重配；若缺少这种跨模型内存协调能力，系统往往难以同时兑现成本收益与时延 SLO [143]。对国产平台而言，这种耦合还会被放大，因为许多后端对低比特算子、动态 shape 和异步搬运的支持边界并不一致。因此，压缩加速的难点并不是有没

有某种量化算法，而是系统能否把“配置参数-执行效率-状态占用-服务质量-单位 token 成本”组织成同一套可观测、可回归、可决策的闭环。如果不能做到这一点，那么压缩策略很容易停留在离线实验中的阶段性结论，难以成为国产推理引擎的稳定生产能力。

进一步说，压缩配置在服务期里不应被理解成一次性静态选择，而更像一类需要随负载形态持续重估的运行策略。相同的低比特权重路径，在短上下文高并发场景中可能主要改善 batch 容量，而在长上下文或多模态场景中则可能首先改变 KV 驻留层级、跨阶段带宽竞争和图模式命中率；同样的 KV 压缩策略，在单轮问答里也许只体现为显存节省，但在多轮会话、Prefix Cache 复用和跨实例迁移下，却会直接改变命中收益是否足以覆盖恢复与搬运代价。因此，国产推理系统真正需要的不是“支持多少种量化格式”，而是能否把压缩策略与请求分类、bucket 组织、缓存层级和回退机制联合决策。当系统知道某类模型、某类上下文长度和某类服务目标下，压缩究竟是在降低单位 token 成本，还是只是在把成本从显存转移到尾时延和恢复路径，它才算真正具备了压缩治理能力。

这也决定了压缩与成本主路径不能继续停留在“离线压缩结果不错，因此线上也应受益”的线性想象上。更可行的优化方向，是把压缩策略重新纳入运行时控制面，包括压缩感知调度能力、量化与 KV 压缩及缓存层级和 batch 组织共享的统一成本画像，以及与回退语义联动的低成本目标治理。原因在于，国产平台经常同时面临算子支持边界、图模式命中条件和显存驻留层级差异，任何一个因素变化，都会改写压缩收益的兑现方式；若系统又无法判断何时应从低比特路径回到更保守路径，服务质量和单位 token 成本都难以稳定经营。

相应地，这一方向更值得持续展开的课题，不是单独比较几种量化格式，而是研究压缩收益如何被在线预测、在线控制和跨模型协同吸收。例如，质量约束下的在线控制器需要同时观察模型效果、吞吐与尾时延，以决定压缩策略应在何种负载下开启或回退；跨模型显存协同则要求系统理解不同模型实例在时间上的显存峰值是否可错开，进而把“多模型共池”从资源口号变成可执行策略；长上下文与多模态场景中的 KV 压缩收益预测，还需要进一步回答状态压缩带来的命中损失和恢复代价何时会反噬吞吐。这些问题如果不能进入统一分析框架，压缩优化就仍然只是离线实验室里的好结果，而不是生产环境中可持续兑现的系统能力。

5.4 统一底座与演进秩序：上游演进与本地分叉维护

高性能推理框架仍在快速演化。国产后端如果通过侵入式修改共享路径获得短期收益，后续将面临高昂的合并成本，因此如何通过插件、注册表、平台抽象和配置门控实现“可合并的本地优化”，就成为工程实践中的核心议题。这一问题的本质，是短期交付与长期演进之间的张力。许多本地优化在初期看似合理，例如直接修改共享 model runner、复制上游模块后做平台特化、或在公共调度路径中快速加入平台分支。这些做法能够尽快打通关键模型，但随着上游版本迭代，它们会迅速演变成难以维护的分叉。国产推理项目真正具有持续价值的能力，不仅是“适配成功”，更是“适配方式本身是否可持续”。从工程组织角度说，本地分叉之所以危险，还因为它会逐步改变团队的优化激励。若某个项目长期依赖复制上游模块并在本地修补，团队最容易积累的是“快速打补丁”的经验，而不是“设计稳定扩展面”的能力；随着时间推移，任何新模型、新特性和新后端都会优先以继续分叉的方式落地，最终导致系统在名义上仍与上游兼容，实则已经丧失合并能力。相反，若平台特化更多收敛在插件、注册表、解析器、模型映射和受控配置门面中，

团队就更有机会把一次性的适配劳动沉淀为可复用的工程机制。以 vLLM-HUST 这类开源主干复用增强实践为例，其价值不只在支持某个具体国产平台，更在于尝试把平台插件链、运行时护栏、多模态注册中心、reasoning parser 和 tool parser 这些高变动能力压在可独立演进的外围接口上 [25]。关键问题不只是版本合并负担，而是国产推理引擎是否在逐步形成自己的模块化演进秩序。若平台适配、模型接入、协议兼容和复杂输出解析都能被压回清晰的扩展边界，后续新增能力就更可能表现为局部增量；反之，如果这些变化都直接冲击共享执行链，那么系统每次升级都会重新暴露整体脆弱性。这也是为什么 merge-safe 并不是“对上游友好”的附属要求，而是国产推理引擎能否持续演进的核心条件。

所谓 merge-safe 并不只是代码目录是否整洁，而是系统是否建立了三层相互约束的演进秩序。第一层是能力声明要稳定，即平台后端、量化路径、图模式、多模态扩展和结构化输出都要先以注册表、配置门面或能力探针显式暴露，而不是靠隐式分支散落在共享路径中。第二层是失败语义要稳定，即回退、降级、禁用和异常恢复必须有统一入口，使新平台或新特性不会各自发明一套错误处理方式。第三层是回归对象要稳定，即每次合并上游后，都能清楚知道哪些模型、哪些执行模式、哪些分布式形态和哪些工具链版本需要重新验证。若缺少这三层秩序，本地分叉带来的代价就不只是 merge 冲突，而是整个团队逐渐失去对系统边界的共同理解。

这一主路径更值得投入的方向，不是继续积累“如何更快打补丁”的经验，而是把平台特化沉淀成稳定扩展面。更具体地说，插件契约、注册表、配置门面和统一失败语义之所以重要，不是因为它们在结构上更整洁，而是因为它们决定了本地优化能否被限制在可预测边界内。一个成熟的国产推理项目，理应能够明确区分哪些变化属于平台能力声明，哪些变化属于运行时策略选择，哪些变化属于服务层模板与治理工具；这样一来，新增模型、新后端和新复杂能力的接入，才不会每次都重新冲击共享执行链。进一步看，升级回归矩阵自动生成的意义也不限于节省人力，它更是在为团队建立一套共同的系统边界认知，使每次跟进上游时都能清楚知道自己究竟在验证什么、承担什么风险。

从研究角度说，这一主路径最值得继续深挖的是“可合并的本地优化”究竟需要怎样的系统条件。merge-safe 插件架构并不是简单的代码组织规范，而是要求平台能力声明、失败语义和回归对象都能够被稳定描述；升级影响分析也不只是比较 diff，而是要识别哪些接口变化会传导到模型映射、调度路径、解析器和复杂输出链路；平台特化与主干接口协同演进机制，则进一步要求团队同时处理交付速度和长期可维护性的矛盾。相比显性的性能数字，这些问题更偏工程秩序，但它们恰恰决定国产推理系统能否从一次次局部适配，转入可持续积累的演进轨道。

5.5 统一底座与证据链：评测体系与真实业务鸿沟

当前不少推理系统的优化仍主要围绕静态 benchmark 展开，例如固定 batch、固定上下文长度或单一模型族上的吞吐对比。但国产推理引擎面向的真实业务往往包含请求长度分布变化大、多租户并发、冷热模型混部、工具调用链条长和 SLA 约束严格等特征。BurstGPT 与针对真实服务流量的工作负载研究已经进一步表明，请求突发性、长度重尾、会话复用和阶段不对称会系统性改变调度、缓存和尾时延表现，因此如果评测体系只覆盖理想化场景，就很难准确反映系统在生产环境中的优势与短板 [111, 113]。因此，未来评测体系需要从“单点性能测试”扩展为“端到端工作负载评测”，将首 token 时延、稳

定吞吐、缓存命中率、异常恢复时间、升级可用性和资源成本等指标纳入统一分析框架。这也是国产推理引擎从工程实现走向体系成熟的重要标志。在这一点上，OpenCompass 与 FlagPerf 分别代表了模型能力评测和系统性能评测两个侧面的生态努力 [144, 145]。前者帮助界定模型在多任务、多语言与复杂能力维度上的效果边界，后者则更强调硬件与系统栈在统一基准上的性能呈现。对于国产推理引擎而言，真正有价值的评测应当把这两类视角连接起来，即既看模型效果，也看支撑这些效果所付出的系统成本。

评测体系的短板并不只是“样本不够真实”，而在于很多关键系统代价缺乏统一表达。例如，长上下文服务是否真正受益于缓存复用，要看命中率、驱逐策略和跨阶段干扰；复杂调度是否有效，要看 SLO 达成率、迁移开销和队列整形效果；多模态 serving 是否可落地，则要看 encode/prefill/decode 三类阶段的资源占用是否被同时纳入统计。Llumnix、SOLA 与 ElasticMM 等工作之所以对综述有价值，正在于它们把动态迁移、SLO 感知与多模态阶段并行这些通常被 benchmark 忽略的系统变量显式化了 [9, 85, 86]。如果国产推理引擎缺乏这类近真实负载的度量框架，就很容易在评测环节高估局部优化、低估长期运维成本。

更进一步，评测体系本身也应当区分不同层次的问题。底层算子和内核优化适合用 microbenchmark 验证，其目的在于确认单点实现是否达到预期；运行时调度、缓存和回退策略需要用受控 serving workload 验证，其重点是吞吐、TTFT、TPOT 与尾时延；而平台演进能力、稳定交付能力和运维代价，则更适合通过持续集成基线、版本升级回归、真实流量回放和故障演练来评估。若这些层次被混在一起讨论，团队就很容易用“单点更快”替代“系统更稳”，用“离线吞吐更高”替代“生产交付更可持续”。国产推理引擎若要真正进入成熟阶段，评测闭环本身也必须从展示型 benchmark 转向决策型 benchmark，即能够直接回答哪一类路线在何种业务约束下更值得继续投入。

据此，综述层面至少需要一个“最小评测协议”作为证据门槛，以避免不同来源的数字在缺少工作负载、版本矩阵和回退语义说明时被直接并列。表14 给出一个适用于国产推理引擎比较的最小协议，它覆盖了从局部 kernel 到端到端交付的三层证据，也对应了前文所谓统一底座应承担的证据组织职责。

这也意味着，国产推理引擎更需要一种分层证据链，而不是单张排行榜。对 backend 和 kernel 路径，应优先回答“局部优化是否在目标硬件上真实成立”；对 runtime 和 serving 路径，应回答“在给定请求分布、上下文长度和复杂能力组合下，系统是否仍能稳定守住 TTFT、TPOT 和尾时延”；对平台交付路径，则应回答“版本升级、模板变更、驱动替换和故障注入后，系统恢复成本是否可控”。只有把这三层证据同时建立起来，路线比较才不会退化为宣传口径拼接。否则，一条路线即使在公开 benchmark 上占优，也可能只是把真实成本留给了环境治理、人工回归和线上排障；另一条路线即使峰值不最激进，只要能稳定提供局部性能证据、运行时证据和交付证据，就更可能成为长期可积累的国产工程底座。

从优化角度看，这一主路径今后最重要的工作，不是继续累积更大的排行榜，而是让评测结果能直接进入架构决策。分层 benchmark 的价值就在于，它要求系统把不同层次的问题分开回答：kernel 和算子层只负责说明局部实现是否成立，运行时层负责说明调度、缓存和回退在受控负载下是否稳定，平台交付层则负责说明升级、回滚和故障恢复是否可控。只有当这三类证据被明确拆开，团队才不会再用单点吞吐替代端到端稳定性，也不会把一次理想化压测的好结果误写成长期可复制的工程结论。真实流量回放、

trace/log/metrics 一体采集和最小评测协议的意义，也都在于把“可展示的结果”进一步改造成“可用于决策的证据”。

若继续上升到课题层面，这里更值得推进的是一类新的评测观：benchmark 从性能展示工具延伸为系统收敛工具。这要求研究者继续补强状态感知指标体系，使多模态前处理、缓存迁移、异常回退和复杂控制流都能被统一表达；也要求跨平台证据格式和真实流量回放协议逐渐成熟，使不同后端、不同路线和不同交付形态可以在相近证据门槛下比较。更进一步，决策型 benchmark 还需要把模型效果、系统成本和交付代价纳入同一分析面，从而直接回答某条路线为何值得继续投入、某种优化为何应被保留、某类 patch 为何不应再扩散。评测闭环具备这种筛选能力后，第五章提出的挑战才会真正转化为后续架构演进的约束与方向。

综上，第五章回答的仍是“问题如何稳定出现、优化抓手落在哪里”这一层次。若把视角再向上抬一层，接下来更需要关心的就不再是单条主路径上还能追加哪些局部修补，而是未来几年哪些研究主线最可能重新组织这些挑战、并决定国产推理引擎的长期收敛方向。

6 未来研究展望：由关键挑战走向长期收敛

第 5 节已将路线比较进一步收束为若干系统挑战，本节则把视角再上移一层，讨论未来几年更值得持续投入的研究方向。与其把“展望”理解为再列举若干可能出现的新功能，不如把它压缩为三条更稳定的主线：面向异构平台的能力契约与共享抽象、面向真实交付的控制面与证据链一体化治理，以及面向复杂请求图的状态中心化运行时。图9 所概括的，正是这三条主线在国产推理生态中的长期收敛关系。



图 9：国产推理引擎未来演进方向关系图

未来演进大致受三组相互牵动的收敛压力支配：一是平台差异能否被沉淀为更稳定的能力边界与共享抽象，二是控制面、评测与交付能否从辅助工具上升为独立治理层，三是模型结构和任务形态变化会如何继续反向塑造系统边界。这些变化不是彼此独立的新功能，而是前文“一个底座、三条主路径”在未来几年内更可能沉淀出的高层研究议题。

这些研究方向并不漂浮在工程现实之外。它们之所以值得在第六章单独提出，恰恰是因为相关压力已经开始倒逼不同主体重写分工：芯片与基础软件提供方需要把平台差异沉淀为稳定能力契约，开源推理框架维护者需要把复杂能力压回可扩展边界，平台交付方需要把控制面、版本矩阵和回归流程产品化，而研究与工程团队则更需要把工作负

载、评测协议与系统证据链对齐。表15 给出的是这三条研究主线落到生态协同时所对应的主体视角路线图。

6.1 面向异构平台的能力契约与共享抽象

第一条研究主线关心的是，平台差异能否从“项目内部经验”上升为“生态可以共享的能力契约”。现有工程摩擦与公开研究共同提示，未来决定差距的未必只是某个热点 kernel 再快几个百分点，而更可能是 graph/eager 约束、通信语义、低比特路径和异常回退条件能否被稳定描述、持续验证并跨项目复用。若这一层抽象迟迟不能成形，国产适配就会长期停留在项目级 patch 与版本试错的循环中。

沿着这条主线继续推进，第 5 节讨论过的“能力契约”“merge-safe 扩展面”和“持续回归对象”就不再只是局部工程经验，而会转化为一组更明确的研究问题：后端能力应如何表达，才能既覆盖平台差异又不把短期特例固化为长期接口；回退语义应如何定义，才能保证图模式切换、缓存状态和分布式执行前后一致；共享抽象又应如何与平台特化协同演进，才能避免主链不断被项目级例外侵蚀。Llumnix、SOLA、LServe、AI Metropolis 与 ElasticMM 等工作之所以重要，也不仅因为它们提供了新的调度或状态机制，更因为它们显示出运行时的组织方式正在脱离“单一模型 serving 外壳”，转向更像资源与状态中枢的形态 [9, 72, 76, 85, 86]。

因此，这条研究主线最终要解决的并不是“谁来多写一些适配代码”，而是如何把“什么由平台保证、什么由引擎吸收、什么由控制面治理”经营成可预测边界。只有当平台差异能够被收敛为共享能力矩阵，而不是反复改写系统主链，新模型、新后端和新任务形态的进入才会从高风险迁移，变成较稳定的生态扩展。

6.2 控制面、评测与交付的一体化治理

第二条研究主线是，控制面能否从运维脚本和部署经验的集合，演化为连接评测、成本与交付的一体化治理层。成本、压缩和评测一旦进入同一闭环，版本矩阵、依赖树、部署模板、灰度发布、故障注入、回归脚本与证据链采集就不再只是帮助引擎跑起来的外围工具，而会越来越接近决定系统是否可交付、可升级、可回滚的核心层。就现有材料看，这些能力一旦较早产品化，局部优化就更容易转化为组织能力。

这条主线之所以构成研究展望，而不只是工程 checklist，在于它要求把原本分散的问题重新抽象成统一对象。压缩收益要想稳定兑现，需要控制面维护策略基线与回退规则；评测要想真正支持决策，需要控制面维持近真实工作负载、版本口径和证据格式；平台适配要想减少人工排障，也需要控制面收纳依赖树、兼容矩阵和环境诊断。作为更偏主链复用与外围治理拆分的开源主干复用增强实践，vLLM-HUST 把平台插件、运行时护栏和环境治理拆开组织的做法，已经显露出“引擎主体 + 控制面工具链”的收敛方向 [25]；而官方适配仓库和平台文档长期强调的版本对齐、PD 分离、CANN/torch_npu 兼容和分布式连通性前置要求，也都在推动控制面从隐性经验变成显性层次 [22, 105]。

这一方向最终会收束为几个更明确的问题：近真实工作负载应如何标准化表达，才能让评测结果进入架构决策；版本基线、回退语义与故障注入应如何统一，才能让升级与灰度成本成为可治理对象；模型效果、系统成本与交付代价又应如何进入同一证据链，才能避免性能结论与生产结论长期脱节。执行面负责 token 生成，控制面负责让这些 token 处于可比较、可复现、可灰度和可回滚的条件之下；只有当控制面被视为与执行面并列

的一等对象，前文讨论的成本、评测与交付问题才更可能从若干局部经验，转化为稳定的生态能力。

6.3 面向多阶段请求图的运行时重构

第三条研究主线指向运行时本身的重构。随着文本 decode、视觉 encode、语音前端、结构化约束验证、工具调用回注和多 Agent 协同被纳入同一服务链路，推理引擎越来越像是在管理一张多阶段请求图，而不再只是维护单一路径的 token 生成循环。对国产推理生态而言，这意味着后端接入、运行时组织、状态治理与交付工具链的分界会继续移动，原本按“模型家族”划定的系统边界也会越来越难维持。

模型侧的变化正在持续改写这张请求图的形状。以 DeepSeek 系列为代表的路线正在把问题从“模型结构更复杂”推进到“模型、推理内核、KV 基础设施与服务运行时是否一体收敛”：DeepSeek-V2/V3 披露的 MLA、MoE 与大规模服务经验，连同 FlashMLA、Mooncake 等配套工作，已经把通信组织、缓存布局、注意力实现和服务基础设施重新推回运行时中心位置 [63, 64, 134, 146, 147]；DeepSeek-V4 则进一步把这种收敛扩展到更明确的软硬协同讨论，公开报告了相关内核在 HUAWEI Ascend NPU 上的验证，并提出面向硬件设计的 co-design 建议 [21]。与此同时，Qwen2-VL、InternVL2、CogVLM2、GLM-4-Voice 与 MiniCPM-o 等模型也说明，视觉、视频和语音链路已经使文本时代的单一路径假设难以维持 [65–67, 129, 131]。这些变化放到第六章中，重点不再是逐项讨论运行时该如何补齐哪种状态治理能力，而是强调未来系统边界会被模型公司、应用形态和硬件路线共同牵引，任何一方的变化都会迫使另外两方重写分工。

从研究问题的角度看，这条主线最终落在三个层面：请求图中的状态转移应如何显式建模，才能避免多模态、工具调用和多 Agent 控制流把复杂度重新散落到外围特性开关中；共享缓存、编译产物与异常回退应如何围绕阶段关系而不是模型名单组织，才能让新任务形态进入时不必重写主链；而运行时又应如何同时容纳模型 co-design、状态治理和交付控制面，才能在复杂负载下维持可预测的系统边界。这个转变未必立刻带来最亮眼的跑分，但更可能决定国产推理系统能否在未来几年内持续吸收新结构、新任务和新平台。

7 结语

本文围绕“生态格局与路线分化—分析骨架与核心架构—基于骨架的路线比较—由路线比较收束出的关键挑战—未来收敛方向”这一主线，对国产算力平台上的大模型推理引擎作了结构化梳理。其核心目的，不是再补一份项目清单，而是把讨论从“谁支持更多模型、谁局部更快”转回到“复杂度被压在哪里、哪些能力已经沉淀为契约、哪些问题仍在共享主链上外溢”这一更稳定的分析层面。

全文表明，评价一条技术路线是否成立，不能只看局部算子、单个模型或一次 benchmark 上的峰值结果，而应看它能否在同一架构中同时承受后端异构、复杂任务负载和上游快速演进三类压力，并在国产芯片、国产基础软件栈和本土部署环境中形成可持续的工程闭环。本文将现有路线概括为开源主干复用增强路线、平台一体化方案与国产原生/独立路线三类，并指出，长期竞争力并不取决于路线名称本身，而取决于平台差异能否被收敛为能力契约，长上下文、多模态、语音和 Agent 负载能否被统一状态机承接，压缩

配置与单位 token 成本能否进入运行时闭环，以及评测与交付工具链能否真实反映生产代价。国产推理引擎下一阶段的竞争重点，也不在单点峰值还能提高多少，而在于谁能更快、更稳、更低成本地吸收模型变化并完成生产交付。对产业侧而言，这意味着建设重点应从围绕单模型做专项优化转向围绕持续演进负载建设系统能力；对研究侧而言，则需要把平台迁移成本、长周期维护成本、复杂工作负载复用性和近真实服务指标放到与性能同等重要的位置。

概括而言，本文的核心判断可进一步压缩为四点：

- (1) 平台异构本身不是问题的终点，真正决定路线可持续性的，是硬件差异能否被收敛为稳定、可验证、可回退的能力契约。
- (2) 长上下文、多模态、结构化输出、工具调用与 Agent 负载的共同约束，正在把推理引擎从 token 生成器推向状态中心化运行时，因而状态治理能力将比局部算子优化更能决定真实上限。
- (3) 量化、KV 压缩与低比特执行只有进入统一的观测、调度和成本闭环，才可能从离线实验结果转化为可持续交付的单位 token 成本优势。
- (4) 国产推理引擎的路线选择，本质上是复杂度分配与责任边界选择：复杂度由 backend、runtime、service state machine、toolchain 还是控制面承担，将直接决定系统的演进速度、交付确定性和长期维护成本。

进一步看，后续研究与工程推进更适合落实为若干能够持续积累的核心问题：平台差异需要被沉淀为稳定、可回退、可验证的能力边界，而不是长期滞留在共享主链中的隐式特判；长上下文、多模态、结构化输出、工具调用与 Agent 负载正在持续改写状态对象的种类与流动方式，因此状态生命周期管理将成为比局部算子优化更稳定的竞争点；压缩收益、评测证据与交付控制面只有进入同一闭环，国产推理系统的成本优势和工程确定性才可能被真实兑现。围绕这些问题持续推进，国产推理引擎的演进才更可能从阶段性适配走向可持续积累。

参考文献

- [1] Woosuk Kwon et al. Efficient memory management for large language model serving with pagedattention. *Proceedings of the ACM Symposium on Operating Systems Principles*, 2023.
- [2] vLLM Team. vllm project. <https://github.com/vllm-project/vllm>, 2024.
- [3] SGLang Team. Sglang project. <https://github.com/sgl-project/sglang>, 2024.
- [4] Gyeong-In Yu et al. Orca: A distributed serving system for transformer-based generative models, 2023. arXiv:2309.06180.
- [5] Amey Agrawal et al. Sarathi-serve: Efficient LLM inference by piggybacking decodes with chunked prefills, 2023. arXiv:2308.16369.
- [6] Yongzhe Zhong et al. Distserve: Disaggregating prefill and decoding for goodput-optimized LLM serving, 2024. arXiv:2401.09670.

- [7] Harsh Patel et al. Splitwise: Efficient generative LLM inference using phase splitting, 2024. arXiv preprint.
- [8] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. Xgrammar: Flexible and efficient structured generation engine for large language models, 2024. MLSys 2025, arXiv:2411.15100.
- [9] Zedong Liu, Shenggan Cheng, Guangming Tan, Yang You, and Dingwen Tao. Elasticmm: Efficient multimodal llms serving with elastic multimodal parallelism, 2025. NeurIPS 2025 Oral.
- [10] 中国信息通信研究院. 大模型推理系统技术与产业白皮书 (2024 年). Technical report, 中国信息通信研究院, 北京, 2024.
- [11] 张峰, 李鑫, 王恩东, et al. 国产算力平台下大模型推理系统技术挑战与发展路径. 计算机学报, 47(10):2345–2368, 2024.
- [12] 崔慧敏, 李广力, 杜臻, et al. 异构算力人工智能推理基础设施的机遇与挑战. 中国计算机学会通讯, 21(7):1–8, 2025.
- [13] Ying Sheng, Zhuohan Li, Lianmin Zheng, et al. Efficient large language model serving: A survey. *arXiv preprint arXiv:2402.01991*, 2024.
- [14] 百度飞桨团队. Fastdeploy 全场景推理部署引擎官方文档. <https://github.com/PaddlePaddle/FastDeploy>, 2025.
- [15] 昆仑芯科技. vllm-kunlun 官方技术仓库. <https://github.com/Kunlunxin/vLLM-Kunlun>, 2024.
- [16] Tsinghua University. 国产大模型推理引擎“赤兔”Chitu 发布. <https://www.tsinghua.edu.cn/info/1182/117628.htm>, 2025. 清华大学新闻稿, accessed 2026-05-07.
- [17] thu-pacman. Chitu 「赤兔」. <https://github.com/thu-pacman/chitu>, 2025. GitHub repository, accessed 2026-05-07.
- [18] xLLM Team. xLLM. <https://xllm.readthedocs.io/zh-cn/latest/>, 2026. 官方文档, accessed 2026-05-07.
- [19] JD Open Source. xLLM-Service. <https://github.com/jd-opensource/xllm-service>, 2026. GitHub repository, accessed 2026-05-08.
- [20] xLLM Team. xLLM technical report, 2025. arXiv:2510.14686.
- [21] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. Repository-local reference material, 2026. Preview technical report.

- [22] vLLM Ascend Team. vllm ascend project. <https://github.com/vllm-project/vllm-ascend>, 2025.
- [23] Cambricon. vLLM-MLU. <https://github.com/Cambricon/vllm-mlu>, 2026. Apache-2.0 GitHub repository, accessed 2026-05-08.
- [24] SGLang Team. Ascend NPUs in SGLang. https://docs.sglang.ai/platforms/ascend_npu.html, 2026. Official platform documentation, accessed 2026-05-08.
- [25] vLLM-HUST Team. vLLM-HUST. [GitHub repository](#), 2026.
- [26] Woosuk Kwon, Sheng Zhu, Aviral Agrawal, et al. vllm: Easy, fast, and cheap llm serving with pagedattention. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [27] 中国信息通信研究院. 大模型推理引擎硬件适配接口规范. <https://www.caict.cn>, 2025.
- [28] 上海人工智能实验室. Deeplink 深度学习异构适配框架技术白皮书. Technical report, 上海人工智能实验室, 上海, 2024.
- [29] 华为技术有限公司. Mindie 大模型推理加速套件技术白皮书. Technical report, 华为技术有限公司, 深圳, 2024.
- [30] Ascend. vllm-ascend 官方技术仓库. <https://gitee.com/ascend/vllm-ascend>, 2024.
- [31] Ascend. Sglang-kernel-npu 官方技术仓库. <https://gitee.com/ascend/SGLang-Kernel-NPU>, 2025.
- [32] Ascend. Triton-ascend 编译后端官方文档. <https://gitee.com/ascend/triton-ascend>, 2024.
- [33] 上海人工智能实验室. Deeplink 深度学习异构适配框架官方仓库. <https://github.com/DeepLink-org/DeepLink>, 2024.
- [34] 上海人工智能实验室. Dlinfer 大模型推理统一适配层技术白皮书. Technical report, 上海人工智能实验室, 上海, 2024.
- [35] 上海人工智能实验室. Lmdeploy 官方技术文档与多硬件适配说明. <https://github.com/InternLM/lmdeploy>, 2025.
- [36] 寒武纪技术团队. vllm-mlu 官方适配仓库. <https://gitee.com/cambricon/vllm-mlu>, 2024.
- [37] 寒武纪技术团队. Magicmind 推理引擎官方技术文档. <https://www.cambricon.com/docs/magicmind/index.html>, 2024.

- [38] 摩尔线程技术团队. vllm-musa 官方适配仓库. <https://github.com/MooreThreads/vllm-musa>, 2024.
- [39] 摩尔线程技术团队. Mt-transformer 大模型推理加速库官方文档. <https://github.com/MooreThreads/MT-Transformer>, 2024.
- [40] 沐曦集成电路. vllm-metax 官方适配仓库. <https://github.com/Metax-Technology/vllm-metax>, 2024.
- [41] 燧原科技. vllm-gcu 官方适配仓库. <https://github.com/Enflame-tech/vllm-gcu>, 2024.
- [42] 燧原科技. Fastdeploy-enflame 官方适配文档. <https://github.com/Enflame-tech/FastDeploy>, 2024.
- [43] 天数智芯技术团队. Ixrt 高性能推理引擎官方技术文档. <https://www.iluvatar.com/docs/ixrt/index.html>, 2024.
- [44] 天数智芯 DeepSpark 生态. Deepsparkinference 大模型推理适配仓库. <https://github.com/DeepSparkHub/DeepSparkInference>, 2024.
- [45] 海光信息技术股份有限公司. Fastdeploy-hygon dcu 官方适配文档. <https://www.hygon.cn/docs/developer/fastdeploy>, 2024.
- [46] 壁仞科技. Birensupa 软件平台官方技术文档. <https://www.birentech.com/developer>, 2024.
- [47] 壁仞科技社区. Enginex-biren vllm 适配项目仓库. <https://github.com/EngineX-Inference/EngineX-Biren>, 2024.
- [48] 瀚博半导体. Vastai 开发者平台官方文档. <https://developer.vastai.com>, 2024.
- [49] 阿里云 Xinference 团队. Xinference 异构推理服务平台官方文档. <https://github.com/xorbitsai/inference>, 2025.
- [50] GPUStack 社区. Gpustack 多硬件推理部署平台官方仓库. <https://github.com/gpustack/gpustack>, 2024.
- [51] 中国电子技术标准化研究院. 人工智能大模型推理引擎技术要求. <https://www.cesi.cn>, 2024.
- [52] 钱学海, 李根, 廖小飞, et al. 面向异构算力的大模型分布式推理系统优化. 软件学报, 35(6):2789–2812, 2024.
- [53] Zirui Liu et al. Kivi: A tuning-free asymmetric 2bit quantization for KV cache, 2024. arXiv:2402.02750.

- [54] Yuhui Li et al. Snapkv: Llm knows what you are looking for before generation, 2024. arXiv:2404.14469.
- [55] Tri Dao et al. Flashattention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, 2022.
- [56] Tri Dao et al. Flashattention-2: Faster attention with better parallelism and work partitioning. *International Conference on Learning Representations*, 2024.
- [57] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024. NeurIPS 2024, arXiv:2407.08608.
- [58] Lei Wang et al. Qserve: W4a8kv4 quantization and system co-design for efficient LLM serving, 2024. arXiv:2405.04532.
- [59] Reiner Pope et al. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 2023.
- [60] Ying Sheng et al. Flexgen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning*, 2023.
- [61] Guangxuan Xiao et al. Streamingllm: Efficient streaming language models with attention sinks, 2023. arXiv:2309.17453.
- [62] LongServe Team. Longserve: Efficient LLM serving for long-sequence requests, 2024. arXiv preprint.
- [63] DeepSeek-AI et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. arXiv:2405.04434.
- [64] DeepSeek-AI. Deepseek-v3 technical report, 2024. arXiv technical report.
- [65] Qwen Team. Qwen2-vl technical report, 2024. arXiv technical report.
- [66] Zhe Chen et al. Internvl2: Better multimodal foundation models with stronger vision-language alignment, 2024. arXiv preprint.
- [67] Aohan Zeng et al. Glm-4-voice: Towards intelligent and human-like end-to-end spoken chatbot, 2024. arXiv:2412.02612.
- [68] Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. Flexattention: A programming model for generating fused attention variants. *Proceedings of Machine Learning and Systems*, 2025.
- [69] Hanqing Jiang et al. Minference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention, 2024. NeurIPS 2024 Spotlight, arXiv:2407.02490.

- [70] Guangxuan Xiao et al. Duoattention: Efficient long-context LLM inference with retrieval and streaming heads, 2024. arXiv:2410.10819.
- [71] Woosuk Kwon et al. vattention: Dynamic memory management for serving LLMs without pagedattention, 2024. arXiv:2405.04437.
- [72] Shang Yang, Junxian Guo, Haotian Tang, Qinghao Hu, Guangxuan Xiao, Jiaming Tang, Yujun Lin, Zhijian Liu, Yao Lu, and Song Han. Lserve: Efficient long-sequence llm serving with unified sparse attention, 2025. MLSys 2025, arXiv:2502.14866.
- [73] Zhenyu Zhang et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models, 2023. arXiv:2306.14048.
- [74] Siyuan Yang et al. Pyramidinfer: Pyramid KV cache compression for high-throughput LLM inference, 2024. arXiv preprint.
- [75] Yibo Zhang et al. Cachegen: KV cache compression and streaming for fast large language model serving, 2024. SIGCOMM 2024, arXiv:2310.07240.
- [76] Zhiqiang Xie, Hao Kang, Ying Sheng, Tushar Krishna, Kayvon Fatahalian, and Christos Kozyrakis. Ai metropolis: Scaling large language model-based multi-agent simulation with out-of-order execution, 2025. MLSys 2025.
- [77] NVIDIA. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>, 2025.
- [78] Microsoft DeepSpeed Team. DeepSpeed-FastGen. <https://github.com/microsoft/DeepSpeed>, 2024.
- [79] FlashInfer Team. FlashInfer. <https://github.com/flashinfer-ai/flashinfer>, 2025.
- [80] NVIDIA. Multimodal support in tensorrt llm. [NVIDIA TensorRT-LLM documentation](#), 2026.
- [81] MIT HAN Lab. Omniserve: Unified and efficient inference engine for large-scale llm serving. [GitHub repository](#), 2025.
- [82] Haichen Zhang, Chang Liu, and Woosuk Kwon. vllm x amd: Efficient llm inference on amd instinct MI300X gpus. [AMD developer technical article](#), 2024.
- [83] Sean Song, David Silverstone, and Mohit Deopujari. Llm inference optimization using amd gpu partitioning. [AMD ROCm blog post](#), 2026.
- [84] Chandrish Ambati and Trung Diep. Amd mi300x gpu performance analysis, 2025. arXiv:2510.27583.

- [85] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving, 2024. OSDI 2024, arXiv:2406.03243.
- [86] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. Sola: Optimizing slo attainment for large language model serving with state-aware scheduling, 2025. MLSys 2025.
- [87] Hugging Face. Transformers. [GitHub repository](#), 2025.
- [88] Ray Team. Ray serve documentation. [Ray Serve documentation](#), 2025.
- [89] NVIDIA. Nvidia triton inference server. [GitHub repository](#), 2025.
- [90] LMSYS Org. Fastchat. [GitHub repository](#), 2025.
- [91] Lianmin Zheng et al. Sglang: Efficient execution of structured language model programs, 2023. arXiv:2312.07104.
- [92] SGLang Team. SGLang ascend NPU development roadmap. <https://github.com/sgl-project/sglang/issues/13664>, 2026. GitHub roadmap issue, accessed 2026-05-08.
- [93] LightLLM Team. Lightllm. <https://github.com/ModelTC/lightllm>, 2025.
- [94] Hugging Face. Text generation inference. <https://github.com/huggingface/text-generation-inference>, 2025.
- [95] MLC Team. Mlc-llm. <https://github.com/mlc-ai/mlc-llm>, 2025.
- [96] Ollama Team. Ollama. <https://github.com/ollama/ollama>, 2025.
- [97] llama.cpp Contributors. llama.cpp. [GitHub repository](#), 2025.
- [98] ExLlama Team. Exllamav2. [GitHub repository](#), 2025.
- [99] mistral.rs Contributors. mistral.rs. <https://github.com/EricLBuehler/mistral.rs>, 2025.
- [100] Aphrodite Team. Aphrodite engine. <https://github.com/aphrodite-engine/aphrodite-engine>, 2025.
- [101] KTransformers Team. Ktransformers. <https://github.com/kvcache-ai/ktransformers>, 2025.
- [102] Predibase. Lorax. <https://github.com/predibase/lorax>, 2025.
- [103] Huawei Technologies. Mindspore. [Official website](#), 2025.

- [104] Huawei Ascend. Compute architecture for neural networks (CANN). [Official website](#), 2025.
- [105] Huawei Ascend. MindIE: Model inference engine. <https://www.hiascend.com/>, 2025.
- [106] FastDeploy Team. FastDeploy documentation. <https://paddlepaddle.github.io/FastDeploy/>, 2026. Official documentation, accessed 2026-05-08.
- [107] LMDeploy Authors. LMDeploy installation. https://lmdeploy.readthedocs.io/en/latest/get_started/installation.html, 2026. Official installation guide, accessed 2026-05-08.
- [108] China Telecom. 中国电信完成业界首个面向国产算力的跨架构大模型推理技术验证. <https://www.chinatelecom.com.cn/ct/news/jtxw/163649.html>, 2025. Official news article on Triton-based cross-architecture LLM inference validation.
- [109] LMDeploy Authors. LMDeploy supported models. https://lmdeploy.readthedocs.io/en/stable/supported_models/supported_models.html, 2026. Official supported-model matrix, accessed 2026-05-08.
- [110] China Telecom. 中国电信总经理刘桂清：从连接到智能云网宽带发展进入新时代. <https://www.chinatelecom.com.cn/ct/news/lddt/165146.html>, 2026. Official speech mentioning Triton heterogeneous LLM inference framework validation.
- [111] Yuxin Wang et al. Towards efficient and reliable LLM serving: A real-world workload study, 2024. arXiv:2401.17644.
- [112] Yongjun He, Yao Lu, and Gustavo Alonso. Deferred continuous batching in resource-efficient large language model serving, 2024. DOI:10.1145/3642970.3655835.
- [113] Yuxin Wang et al. Burstgpt: A real-world workload dataset to optimize LLM serving systems, 2025. DOI:10.1145/3711896.3737413.
- [114] PaddlePaddle. FastDeploy. <https://github.com/PaddlePaddle/FastDeploy>, 2026. High-performance inference and deployment toolkit for LLMs and VLMs, accessed 2026-05-07.
- [115] OpenMMLab. LMDeploy. <https://github.com/InternLM/lmdeploy>, 2025.
- [116] LightLLM Team. LightLLM documentation. <https://lightllm-en.readthedocs.io/en/stable/>, 2026. Official documentation, accessed 2026-05-08.
- [117] Ruihao Gong, Shihao Bai, Siyu Wu, Yunqian Fan, Zaijun Wang, Xiuhong Li, Hailong Yang, and Xianglong Liu. Past-future scheduler for LLM serving under SLA guarantees. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

- [118] Qingcheng.AI. 赤兔 Chitu. <https://www.qc-ai.cn/products/chitu>, 2025. 清程极智产品页面, accessed 2026-05-07.
- [119] TsingmaoAI. 玄武 XuanWu: 国产算力的大模型自由. <https://xw.tsingmao.com/>, 2026. 官方产品页面, accessed 2026-05-07.
- [120] Lequn Chen et al. Punica: Multi-tenant lora serving, 2023. arXiv:2310.18547.
- [121] S-LoRA Team. S-lora: Serving thousands of concurrent lora adapters, 2024. arXiv preprint.
- [122] Guidance Team. Guidance. <https://github.com/guidance-ai/guidance>, 2025.
- [123] Outlines Team. Outlines. <https://github.com/dottxt-ai/outlines>, 2025.
- [124] MLC Team. Xgrammar. <https://github.com/mlc-ai/xgrammar>, 2025.
- [125] PaddlePaddle. PaddleNLP LLM Server. <https://github.com/PaddlePaddle/PaddleNLP/tree/develop/llm/server>, 2026. PaddleNLP repository LLM service entry, accessed 2026-05-07.
- [126] Qwen Team. Qwen2-vl. <https://github.com/QwenLM/Qwen2-VL>, 2025.
- [127] OpenGVLab. Internvl. <https://github.com/OpenGVLab/InternVL>, 2025.
- [128] THU DM. Cogvlm2. <https://github.com/THU DM/CogVLM2>, 2025.
- [129] Weixiong Wang et al. Cogvlm2: Visual language models for image and video understanding, 2024. arXiv preprint.
- [130] Yunfei Chu et al. Qwen2-audio technical report, 2024. arXiv:2407.10759.
- [131] Junbo Cui et al. Minicpm-o 4.5: Towards real-time full-duplex omni-modal interaction, 2026. arXiv:2604.27393.
- [132] Gursimran Singh, Xinglu Wang, Yifan Hu, Timothy Yu, Linzi Xing, Wei Jiang, Zhefeng Wang, Xiaolong Bai, Yi Li, Ying Xiong, Yong Zhang, and Zhenan Fan. Efficiently serving large multimodal models using EPD disaggregation, 2025. ICML 2025, arXiv:2501.05460.
- [133] Zicong Hong, Yuyan Chen, Haoyue Zhang, Peng Li, Wuhui Chen, Song Guo, and Xiaowei Shen. Efficient multimodal serving via module multiplexing, 2026. EuroSys 2026.
- [134] DeepSeek-AI. Flashmla. [GitHub repository](#), 2025.
- [135] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable, 2024. OSDI 2024, arXiv:2405.19888.

- [136] Xupeng Miao et al. Specinfer: Accelerating generative large language model serving with speculative inference and token tree verification, 2023. [arXiv:2305.09781](#).
- [137] Yaniv Leviathan et al. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning*, 2023.
- [138] Tianle Cai et al. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. [arXiv:2401.10774](#).
- [139] Yuhong Li et al. Eagle: Speculative sampling requires rethinking feature uncertainty, 2024. [arXiv:2401.15077](#).
- [140] Yichao Fu et al. Break the sequential dependency of LLM inference using lookahead decoding, 2024. [arXiv:2402.02057](#).
- [141] Sonali Singh, Karthik Sangaiah, Shenrun Zhang, Ryan Swann, and Ganesh Dasika. Accelerating llm inference: Up to 3x speedup on MI300X with speculative decoding. [AMD ROCm blog post](#), 2025.
- [142] Mohammad Mahdi Kamani, Parsa Fashi, Vikram Appia, and Emad Barsoum. Beyond text: Accelerating multimodal ai inference with speculative decoding on AMD instinct MI300X gpus. [AMD ROCm blog post](#), 2025.
- [143] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, Ion Stoica, Harry Xu, and Ying Sheng. Prism: Unleashing gpu sharing for cost-efficient multi-llm serving, 2025. [arXiv:2505.04021](#).
- [144] OpenCompass Team. Opencompass. [GitHub repository](#), 2025.
- [145] FlagOpen. Flagperf. [GitHub repository](#), 2025.
- [146] Xupeng Qin et al. Mooncake: A KVcache-centric disaggregated architecture for LLM serving, 2024. [arXiv:2407.00079](#).
- [147] Xupeng Miao et al. Mooncake store: A KVcache-centric distributed store for LLM serving, 2024. technical report.

表 4: 国产推理引擎常用指标的统一口径

指标/对象	统一定义	使用边界与解释重点
TTFT	请求提交到首个可见 token 返回的时间，通常覆盖排队、前处理、prefill 与首次采样	适合衡量在线交互体验；若比较跨平台 TTFT，应同时说明 prompt 长度、batch 组织、是否包含多模态前处理与图编译预热
TPOT	稳态生成阶段相邻两个输出 token 之间的平均时间	主要反映 decode 主路径效率；应与 batch 大小、采样策略和 speculative decoding 配置一并解释，避免脱离负载单独比较
吞吐/goodput	单位时间完成的 token、请求或满足 SLO 的有效工作量	吞吐适合描述系统上限，goodput 更适合描述真实服务收益；若只报告吞吐，容易掩盖尾时延和回退开销
KV 命中率/迁移开销	前缀复用成功比例，以及状态跨层级、跨实例或跨节点搬移所付出的流量与时延	适合评估长上下文、多轮会话和 PD/EPD 路径；必须结合请求分布、缓存策略和驱逐规则解释
图模式命中率/回退率	请求在 graph 模式稳定执行的比例，以及回退到 eager 或其他安全路径的频率	适合评估 shape 敏感后端的稳定性；若不同时给出回退原因，单独的图命中率意义有限
单位 token 成本	在既定质量和 SLO 约束下，生成单位 token 所需的综合资源成本	不应被理解为静态硬件单价，而应同时吸收量化配置、状态容量、迁移流量、实例利用率与运维开销
可复现性证据	支撑结论的代码、配置、版本矩阵、trace、日志与工作负载描述是否完整	不是单一性能指标，而是路线比较的证据门槛；缺少该项时，结论应降格为趋势判断或工程观察

表 5: 基于开源主干复用的增强路线组件分层与国产算力潜在优化点

组件层	主要职责	可结合国产算力的潜在优化方向
入口与启动护栏	CLI、OpenAI 兼容服务、启动时序控制、运行时预检与环境初始化	针对国产驱动/编译链构建更稳定的启动护栏，在引擎启动前完成 NPU 预检、环境变量自动补齐、依赖矩阵校验与故障前移，减少平台问题侵入执行主路径
配置与能力装配	参数收敛、统一配置对象组装、不同能力开关与默认值选择	为国产后端提供平台感知的默认配置策略，如图模式开关、bucket 粒度、KV 容量预算、低比特路径和分布式后端选择，使优化从零散参数经验转为可复用配置模板
引擎编排与 EngineCore	请求对象转换、流式生命周期管理、调度与 core 通信	面向 shape 敏感后端优化连续批处理、prefill/decode 解耦、SLO 感知调度与前缀局部性管理，并在多进程/多实例形态下减少编译抖动和跨阶段干扰
模型执行与热点算子	模型注册、权重加载、worker/runner、attention 与量化等热点路径	围绕国产硬件补齐 fused attention、MLA/MoE 相关算子、低比特 kernel、图安全回退路径和异步搬运策略，把平台特征转化为稳定吞吐和时延收益
平台插件与运行时治理	平台探测、插件注入、后端激活、环境修复与外围治理工具衔接	把国产平台差异收敛为能力契约，通过插件化方式承载 dtype、attention backend、compile backend、distributed backend 与 runtime 修复逻辑，降低对共享主干的侵入深度
多模态、Reasoning 与 Tool 横切层	多模态注册、IO 处理、reasoning parser、tool parser、复杂输出协议适配	把视觉 encoder、语音前端、结构化输出与工具调用状态统一纳入生命周期管理，并结合国产后端对 encode/decode 分段执行、跨模态缓存和 parser 前后处理开销做协同优化
编译、KV Cache、分布式与 tracing 基础设施	graph/compile、缓存管理、并行执行、监控与链路追踪	针对国产编译栈强化 graph cache 命中、piecewise backend 选择、KV 驻留层次、通信路径调优与近真实负载观测，使性能优化、容量管理和运行时诊断形成闭环

表 6: 国产算力推理路线的复杂度落点

技术路线	代表对象	复杂度主要落点	对国产算力适配的关键意义
插件式后端适配	vLLM-Ascend、vLLM-MLU、SGLang Ascend[22–24]	plugin backend、attention backend、分布式运行时、版本矩阵	以较低上层接口改动把 Ascend NPU、寒武纪 MLU 等国产硬件纳入 vLLM/SGLang 语义，但成熟度受 CANN、torch_npu、厂商 SDK 与上游版本节奏约束
平台一体化	MindIE[105]	SDK/runtime/compiler、服务模板、监控与交付链路	把 Ascend 平台的编译、运行时和服务化交付收敛到平台栈内部，适合强调统一交付的场景，但不宜写成完全开源的通用引擎
国产独立引擎	Chitu、xLLM[16–20]	service state machine、PD/EPD、KV Cache、低精度与多硬件部署闭环	说明国产推理系统不只是在主流框架中补后端，也可以围绕状态治理、缓存治理和部署闭环重构引擎边界
工具链型部署	FastDeploy、LMDeploy[106, 107, 109]	模型接入、OpenAI/vLLM 风格接口、量化路径、硬件安装矩阵	将国产算力适配沉淀为“部署工具链 + 服务接口 + 硬件适配层”，但不同后端和不同硬件上的 feature parity 需要逐项核验
跨架构编译/运行时探索	中国电信 Triton 跨架构框架 [108, 110]	compiler/runtime 抽象、统一算子库、透明运行时插件	试图缓解多国产芯片重复适配问题；当前更适合作为跨架构抽象探索，不能写成已开源成熟 serving 引擎

表 7: 结合公开技术资料的代表对象证据画像

代表对象	公开资料中更早暴露的工程前提	公开资料中可见的工程摩擦	公开资料支持的系统收益或边界	对应的复杂度主要落点
vLLM-Ascend	默认承接 vLLM/OpenAI 兼容接口与上游服务语义, 工程入口建立在主干与插件协同之上	上游主干、插件分支、CANN 与 torch_npu 需要严格对齐; 分布式配置、PD 分离和环境预检经常构成首轮门槛 [22]	以较低上层改动成本验证国产迁移路径, 较快吸收上游 serving 特性, 但高级能力成熟度仍需逐项核验	backend 适配、版本矩阵与环境治理
MindIE	默认将镜像、模板、监控与服务组件一起纳入平台交付路径	配置参数、服务模板、平台组件依赖和网络环境准备对部署成功率影响很大 [105]	公开材料更稳定支持其交付责任集中、运维路径清晰和平台确定性较强的特征	platform runtime、模板治理与交付链路
Chitu	公开资料把其描述为围绕独立生产级引擎组织多元算力兼容、弹性部署与企业落地能力 [16, 17, 118]	独立引擎需要同时经营模型支持矩阵、服务接口和交付模板, 若模块边界不清晰, 演进成本会快速上升	更容易把多硬件兼容、生产可用性和部署弹性放到同一套自研内核里统筹	service state machine、部署闭环与模型矩阵
xLLM / 玄武	公开入口强调围绕国产芯片同步建设推理框架与部署入口 [18, 119]	需要同时维护推理内核、分布式执行与部署入口的一致性, 避免把入口便利性建立在脆弱环境假设上	更容易形成面向国产芯片的“框架 + 部署入口”一体化落地路径	推理内核、部署入口与服务闭环
LMDeploy	文档同时暴露 TurboMind 与 Py-TorchEngine 两条路径, 国产平台扩展更多通过后者进入 [107, 109, 115]	国产 GPU 适配成熟度、量化路径和不同硬件上的最佳实践需要逐平台核验, 模型与设备支持粒度必须按官方矩阵核查	部署效率和高性能工具链较平衡, 但 feature parity 明显依赖具体后端与平台组合	工具链、模型支持矩阵与硬件安装路径
SGLang	公开资料将复杂控制流、长上下文前缀复用和结构化输出放在运行时中心位置 [24]	国产化部署入口当前更依赖后端适配、运行模式说明和执行模型理解, 工程门槛更多落在负载理解与执行模式选择上	在复杂控制流、长 system prompt 复用和结构化输出场景下更容易体现运行时组织优势	运行时控制流、调度抽象与后端适配
开源主干复用增强路线中的本土增强实践 (以 vLLM-HUST 为例)	公开资料显示其保持 vLLM 上层接口, 同时把平台护栏、插件与治理工具拆到共享主链外侧 [25]	需要持续经营模块边界, 避免既继承上游复杂度又堆积本地特化补丁; 对工程架构能力要求更高	在接口兼容、平台特化、AGI4S 场景增强和运行时治理之间形成可持续折中	主干外侧治理层、插件契约与 merge-safe 演进

表 8: 典型工作负载下更先暴露的系统约束与路线承载重心

工作负载	更先暴露的系统矛盾	公开资料中更常承载该矛盾的路线	首先要核验的能力	常见失稳来源
短文本高并发在线生成	decode 带宽、连续批处理效率、版本矩阵与服务稳定性	主线后端适配与工具链强化路线中最常被首先观察到	OpenAI 兼容接口、连续批处理、图模式稳定性、量化收益是否可复现	容易只看到吞吐而忽视尾时延、回退频率和环境依赖脆弱性
长上下文知识问答与文档检索增强	Prefix Cache、Chunked Prefill、状态迁移与单位 token 成本	开源主干复用增强路线与国产独立引擎路线中更常显化	前缀命中率、分层 KV Cache、PD/EPD 解耦、长上下文稳定性	若缓存与调度未联动，容量节省可能转化为恢复时延和队列抖动
多图文档理解与视频问答	encode/prefill/decode 阶段异质性、多模态前缀复用与 shape 波动	本土增强实践、一体化路线与国产独立引擎路线更常承担	视觉前处理是否进入生命周期管理、跨阶段资源再平衡、图模式回退语义	只补前处理适配器而不重写状态机时，模型可运行但服务表现不稳定
结构化输出与工具调用 Agent	约束状态、外部回注、复杂控制流与异常恢复	复杂控制流路线、本土增强实践与国产独立引擎路线更常首先暴露	grammar/tool 状态能否进入调度闭环、回退路径是否统一、日志与 tracing 是否完整	若运行时只面向 token 生成设计，复杂能力会持续挤压共享主链并放大维护成本
固定业务模板的政企交付	交付责任集中、监控模板、版本收敛与问题定位效率	平台一体化路线中最常被作为首要约束处理	镜像、模板、监控、回滚和灰度能力是否成套提供	平台确定性较强，但新模型和新特性的吸收速度通常慢于主干复用路线
多硬件资源池与长期自研平台	多后端兼容、状态中心化运行时、部署入口与演进秩序	国产独立引擎路线与开源主干复用增强中的本土增强实践更常长期承受	service state machine、缓存治理、插件契约、merge-safe 演进机制	初期建设成本高，若边界经营失败，容易同时承担接口、模型矩阵和交付闭环的维护负担

表 9: 国内部署路线案例比较

代表对象	路线类型	工程落点	部署目标
vLLM-Ascend[22]	主线后端适配	围绕上游接口扩展后端、执行器与定制算子	快速跟进上游能力，优先保障生态兼容与模型覆盖
vLLM-MLU[23]	主线后端适配	依托 vLLM plugin system 接入寒武纪 MLU，并覆盖 Chunk Prefill、Prefix Caching、Spec Decode 等 vLLM 能力	以继承 vLLM 接口与调度语义为主，但复现性仍受 SDK/runtime 与公开 benchmark 粒度约束
SGLang Ascend[24, 92]	主线后端适配	在 SGLang 主线中为 Ascend NPU 提供 attention backend、PD 分离部署和组件版本映射	体现国产 NPU 被主流 serving 框架吸纳，但高级能力成熟度需与 roadmap 区分
vLLM-HUST[25]	开源主干复用增强中的本土增强实践	保持 vLLM 上层接口，并注入国产平台插件、AGI4S 场景能力与运行时护栏	兼顾上游兼容、国产平台接入与部署运维闭环
MindIE[105]	一体化部署	与本地编译器、运行时和交付链路深度协同	平台统一交付、稳定运行和行业方案落地优先
Chitu「赤兔」[16, 17, 118]	独立自研引擎	面向多元算力组织生产级推理、弹性部署与企业落地能力	同时重视多硬件兼容、部署弹性与生产可用性的场景
xLLM / 玄武[18, 119]	自研框架 + 部署入口	围绕国产芯片组织开源智能推理框架，并以前端部署入口补齐一键拉起、OpenAI 兼容与预验证能力	希望同步建设推理内核与部署闭环的国产化落地场景
LMDeploy[115]	工具链强化	围绕模型支持、推理工具链和工程化部署体验持续扩展	兼顾高性能部署、模型覆盖和工程效率
FastDeploy / PaddleNLP Server[114, 125]	工具链强化 + 服务封装	以高性能推理与部署工具包衔接模型导出、服务封装和应用接入	希望围绕飞桨生态组织 LLM/VLM 推理与部署链路的场景
中国电信 Triton 统一跨架构框架 [108]	跨架构编译/运行时	通过 Triton 跨架构编译器、统一算子库和 vLLM-Triton 透明插件探索多芯迁移	用于缓解多芯重复适配问题，但未开源且许可证与复现脚本未明确

表 10: 国内团队多模态推理链路工程关注点比较

代表模型	主要链路	关键工程压力	对推理引擎的直接要求
Qwen2-VL、InternVL2[65, 66]	多图、文档与 OCR	动态分辨率切图、视觉 token 膨胀、图文混排带来 shape 波动	视觉前处理与文本 prefill 协同、前缀缓存复用、bucket 化与编译抖动控制
CogVLM2[129]	视频理解	帧采样与跨帧压缩策略影响 encode 与 decode 资源分配	视觉 encoder 批处理、跨阶段资源再平衡、视频前缀组织
Qwen2-Audio 等 [67, 130, 131]	语音与实时交互	音频前端、tokenizer、流式 chunk 与打断恢复共同影响时延	流式调度、语音文本交错生成、实时率控制与会话状态恢复

表 11：国产推理引擎关键挑战比较

挑战来源	主要冲击层	典型后果
硬件异构与后端碎片化	执行层、算子接口、通信路径	共享路径分叉、正确性边界不一致、维护成本持续上升
复杂任务与多模态负载	调度、缓存、图模式与前处理链路	阶段争用加剧、时延波动放大、局部功能可用但端到端不稳定
压缩收益与系统成本错配	量化路径、KV 状态、调度与成本核算	低比特收益不稳定、精度与时延相互牵制、单位 token 成本难形成闭环
上游快速演进与本地分叉	平台抽象、插件接口与共享模块	patch 累积、合并困难、对新模型和新特性的跟进滞后
评测体系与真实业务脱节	benchmark 设计、运维指标与资源评估	优化方向失真、离线结论与生产表现偏离

表 12: 不同国产推理路线与关键挑战的映射关系

代表路线	版本矩阵/环境门槛	网络与分布式风险	缓存/调度压力	运维与回退焦点	更先暴露的能力缺口
vLLM-Ascend 类主线后端适配	高。上游主干、插件分支、CANN 与 torch_npu 对齐要求强，环境预检缺失时很容易在启动阶段失败 [22]	中到高。PD 分离、分布式连通性和阶段解耦常在扩容时暴露问题 [22]	高。更容易先沿用上游连续批处理与 KV 组织，再逐步补本土平台约束，长上下文和复杂 shape 下调度抖动更敏感	高。需要围绕环境校验、插件失配、图模式回退与服务启动护栏建立可观测修复路径	平台能力探测、版本矩阵校验、分布式连通性检查和主链外侧治理工具
MindIE 类一体化方案	中。更多复杂度被平台镜像、模板和组件版本收敛，但外部团队对底层差异的显式控制较弱 [105]	中。网络与服务编排问题更多表现为平台部署参数、服务组件协同与交付流程约束	中。缓存、调度与编译优化更容易被平台内部统一处理，但也更依赖平台默认策略与产品节奏	高。监控、配置模板、回滚路径与交付责任集中，是一体化路线真正的成败点 [105]	模板化部署、监控告警、配置基线、统一回归与平台级问题定位闭环
LMDeploy 类工具链强化路线	中。模型支持矩阵、量化路径和不同硬件上的最佳实践需要逐平台核验 [109, 115]	中。分布式风险通常不先表现为链路不可用，而是表现为不同部署形态下性能与稳定性的权衡不透明	中到高。工具链强调高性能部署与模型接入效率，但在国产 GPU/信创环境中，缓存策略、量化收益和调度参数常需要额外试配	中到高。公开部署资料更常把焦点放在部署脚本、量化回退、多模型兼容和工程试验效率上 [107, 109, 115]	面向不同平台的性能基线、量化/非量化双路径、模型接入自动化和快速回归脚本
开源主干复用增强路线中的本土增强实践（以 vLLM-HUST 为例）	中到高。既要跟上上游接口变化，又要维持插件、护栏和治理层的边界稳定 [25]	中到高。网络与分布式问题不仅来自链路本身，还来自哪些能力留在主链、哪些能力下沉到外围治理层的架构选择	高。要同时承接上游 serving 主体与本地化能力扩展，缓存、调度、多模态和复杂控制流更容易形成横向耦合	高。若模块边界经营不好，最容易同时继承上游复杂度与本地特化补丁负担	模块化扩展面、插件契约、外围治理工具、复杂能力注册中心与 merge-safe 演进机制

表 13: 关键挑战对应的优化抓手与课题方向

挑战主路径	更直接的工程优化抓手	可进一步展开的课题方向
执行与通信	能力契约化后端、graph/eager 混合执行、拓扑感知通信、版本矩阵自动校验、异常回退分层治理	条件化能力 IR、图安全回退语义、跨平台 collective 规划、国产后端 feature parity 的持续回归机制
状态治理	统一请求状态机、阶段 DAG 调度、多模态前缀缓存、分层 KV Cache、状态迁移与回收策略联动	状态中心化运行时、多模态缓存计价、工具调用与结构化输出状态接口、异构资源下的 PD/EPD 调度
压缩与成本协同	压缩感知调度、量化与缓存布局联合决策、在线成本画像、回退感知低比特路径选择	质量约束下的在线控制器、跨模型显存协同、长上下文 KV 压缩收益预测、成本与 SLO 统一优化
演进秩序与扩展边界	插件契约、注册表、配置门面、统一失败语义、升级回归矩阵自动生成	merge-safe 插件架构、能力声明语言、升级影响分析、平台特化与主干接口协同演进机制
评测与证据链	分层 benchmark、真实流量回放、最小评测协议、trace/log/metrics 一体采集	决策型 benchmark、状态感知指标体系、跨平台证据格式、从局部 kernel 到端到端交付的闭环评测

表 14: 国产推理引擎比较的最小评测协议

评测层次	至少应报告的内容	主要回答的问题	缺失时的风险
局部内核与算子层	硬件代际、精度配置、输入 shape、算子版本、microbenchmark 条件	某个 kernel、图优化或低比特路径是否在目标平台真实成立	容易把局部最佳结果误写成系统级优势，且无法判断是否可迁移到 serving 主路径
运行时与 serving 层	模型版本、输入/输出长度分布、并发度、batch 策略、TTFT、TPOT、吞吐、尾时延、回退率	在给定负载下系统是否能稳定组织 prefill/decode、缓存和调度	容易用平均吞吐掩盖 tail latency、图模式失稳和异常恢复成本
状态与缓存层	Prefix Cache 命中率、KV 容量预算、迁移流量、驱逐策略、长上下文配置	状态治理是否真正支撑长上下文、多轮会话和 PD/EPD 路径	容易把显存节省与真实收益混淆，忽略恢复、搬运和命中失败代价
复杂能力层	多模态前处理路径、结构化输出约束、tool/reasoning 回注、异常回退语义	复杂请求是否被纳入统一生命周期管理，而不是局部功能拼接	容易出现“功能可用但端到端不稳”的伪成熟结论
平台与交付层	驱动/SDK 版本、容器镜像、依赖矩阵、部署模板、监控告警、回滚与故障恢复流程	路线是否具备可持续交付、回归与排障能力	容易把人工经验维持的可运行状态误写成可复制的工程能力
证据完整性层	代码入口、配置片段、脚本、trace/log 样例、是否开源、是否可复现实验	结论属于可验证证据、官方验证口径，还是趋势性工程观察	不同性质的材料被强行同列，导致路线比较退化为宣传口径拼接

表 15: 三条研究主线在国产推理生态中的主体分工与演进重点

主体	更应优先收敛的能力	近期更现实的建设重点	中期更关键的演进目标
芯片厂商与基础软件栈提供方	设备能力表达、编译/运行时边界、通信语义与低比特路径稳定性	减少版本矩阵碎片化, 明确 graph/eager、attention backend、通信后端和量化 kernel 的可用边界	把平台差异沉淀为可验证的能力契约, 而不是持续依赖项目级特判和临时 patch
开源推理框架维护者	插件接口、模型执行抽象、复杂能力扩展面与回退语义	为国产后端保留更稳定的 backend、cache、compile、tool/reasoning 扩展点, 降低共享主链分叉压力	让多模态、结构化输出、Agent 与长上下文能力在主线抽象内持续演进, 而不是被外围分支各自重写
平台交付方与云边部署团队	控制面、灰度回滚、监控告警、版本基线和故障注入流程	把镜像、驱动、依赖树、部署模板和回归脚本产品化, 降低“能跑起来”对人工经验的依赖	形成统一的交付控制面, 使性能回归、环境稳定性和单位 token 成本进入同一治理流水线
研究团队与工程集成团队	工作负载建模、评测协议、状态治理策略与证据链组织	用近真实 serving workload 替代单一展示型 benchmark, 补齐 TTFT、TPOT、KV 命中率、回退率等指标口径	建立能连接模型效果、系统成本和交付代价的分层评测体系, 使路线比较真正支持架构决策
模型公司与上层应用构建者	模型结构变化对运行时边界的约束表达、复杂请求生命周期抽象	在发布新模型能力时同步暴露对缓存、调度、约束解码和多阶段执行的系统需求	推动模型设计、推理内核与服务运行时更紧密 co-design, 避免系统复杂度全部滞后暴露在部署阶段